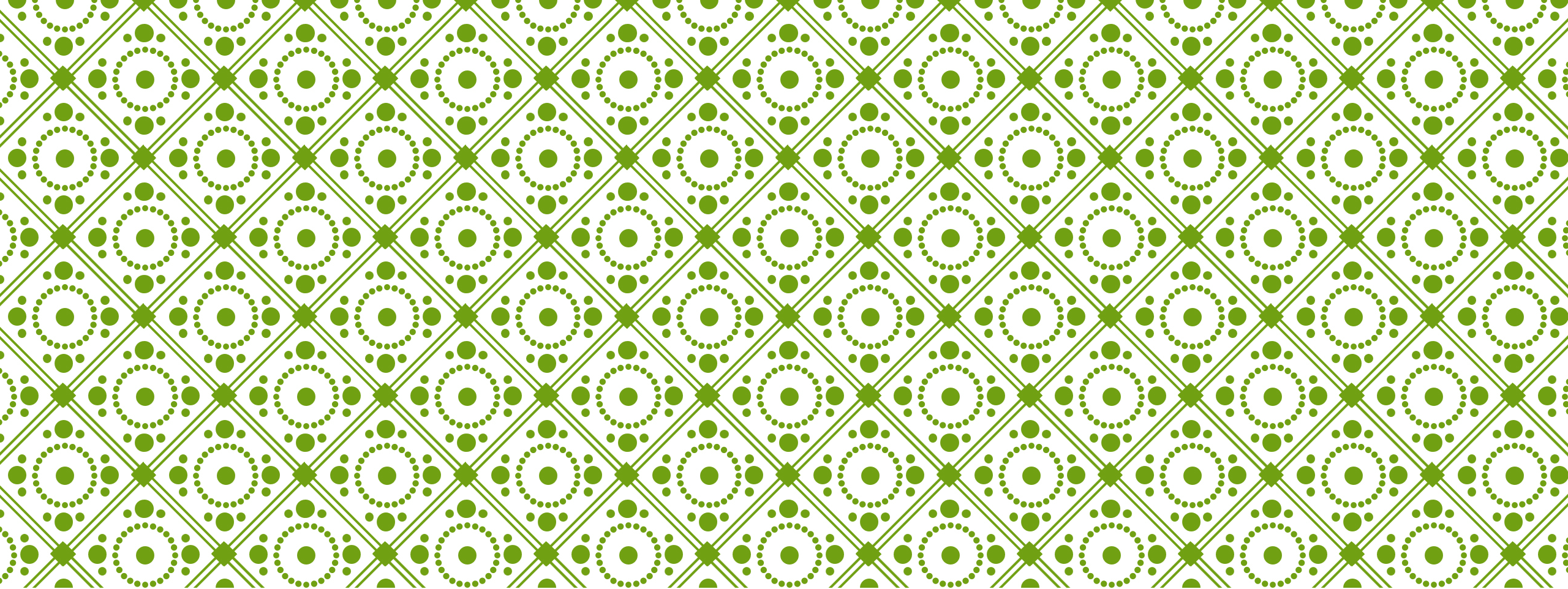




AI2002 — ARTIFICIAL INTELLIGENCE

SPRING 2024 - LECTURE 07-09

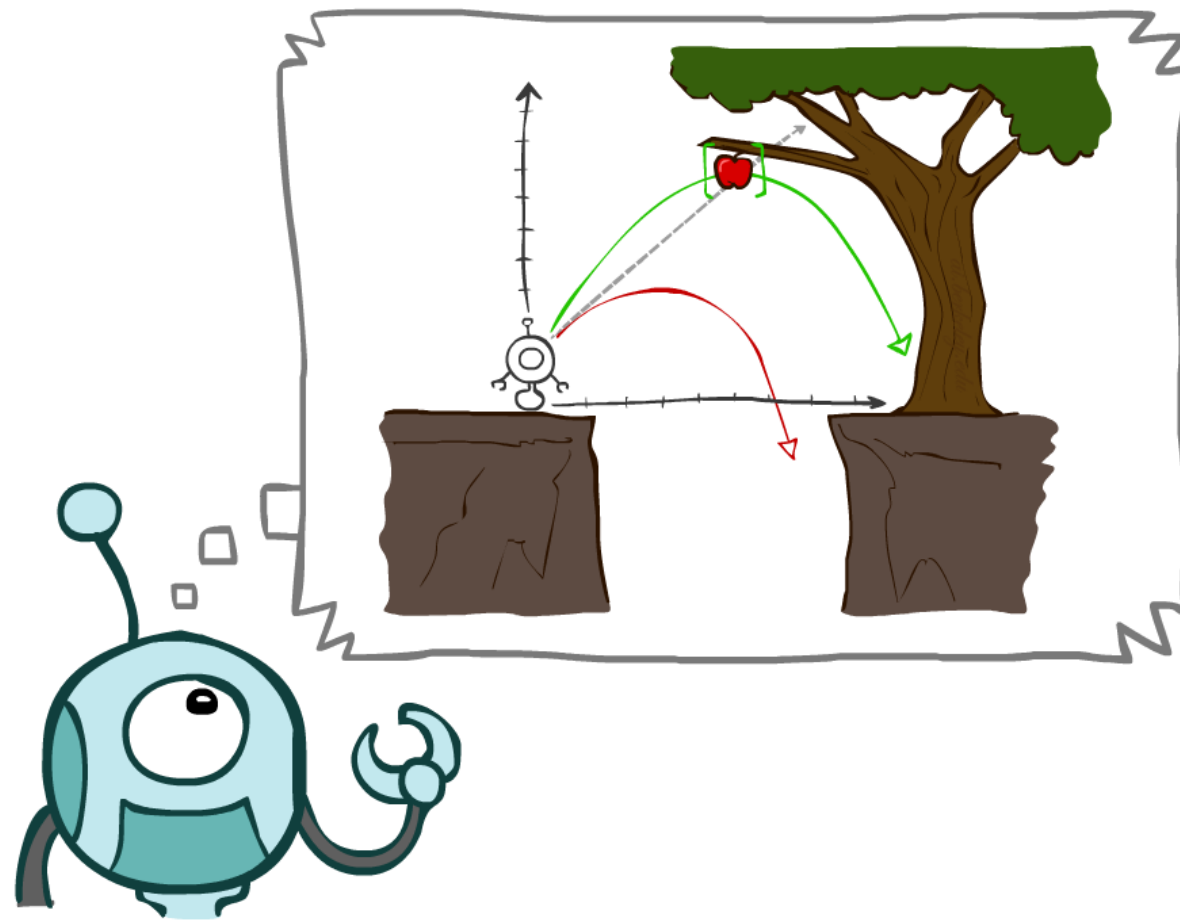
Presented By: Mr. Sandesh
Kumar

Few slides are taken from UC
Berkeley

Problem-Solving Agent

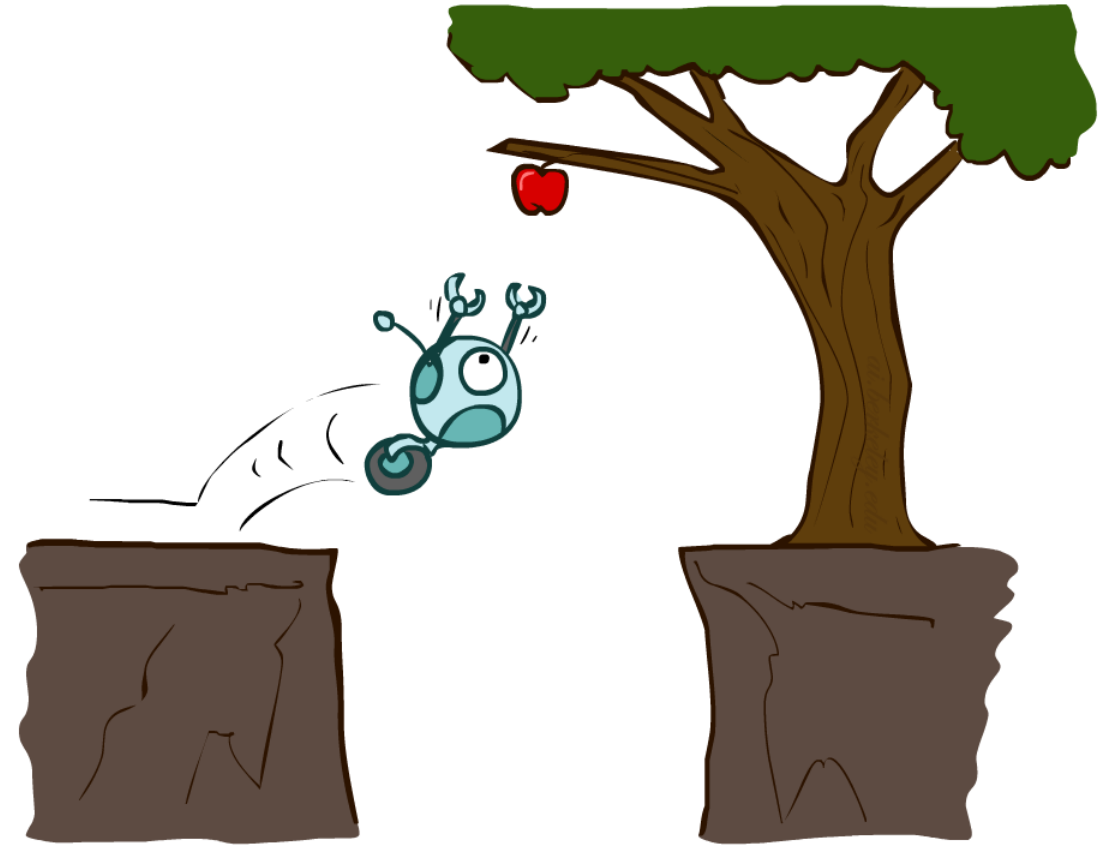
- A **problem-solving agent** is a *goal-based agent* and use *atomic representations*.
 - In atomic representations, states of the world are considered as wholes, with no internal structure visible to the problem solving algorithms.
- *Intelligent agents* are supposed to maximize their *performance measure*. Achieving this is sometimes simplified if the agent can adopt a **goal** and aim at satisfying it.
- **Problem formulation** is the process of deciding what actions and states to consider, given a *goal*.
- The process of looking for a sequence of actions that reaches the goal is called **search**.
- A **search algorithm** takes a problem as input and returns a **solution** in the form of an action sequence.
- Once a *solution* is found, the carrying actions it recommends is called the **execution phase**.
- A problem-solving agent has three phases:
 - **problem formulation, searching solution and executing actions in the solution.**

Agents that Plan



Reflex Agents

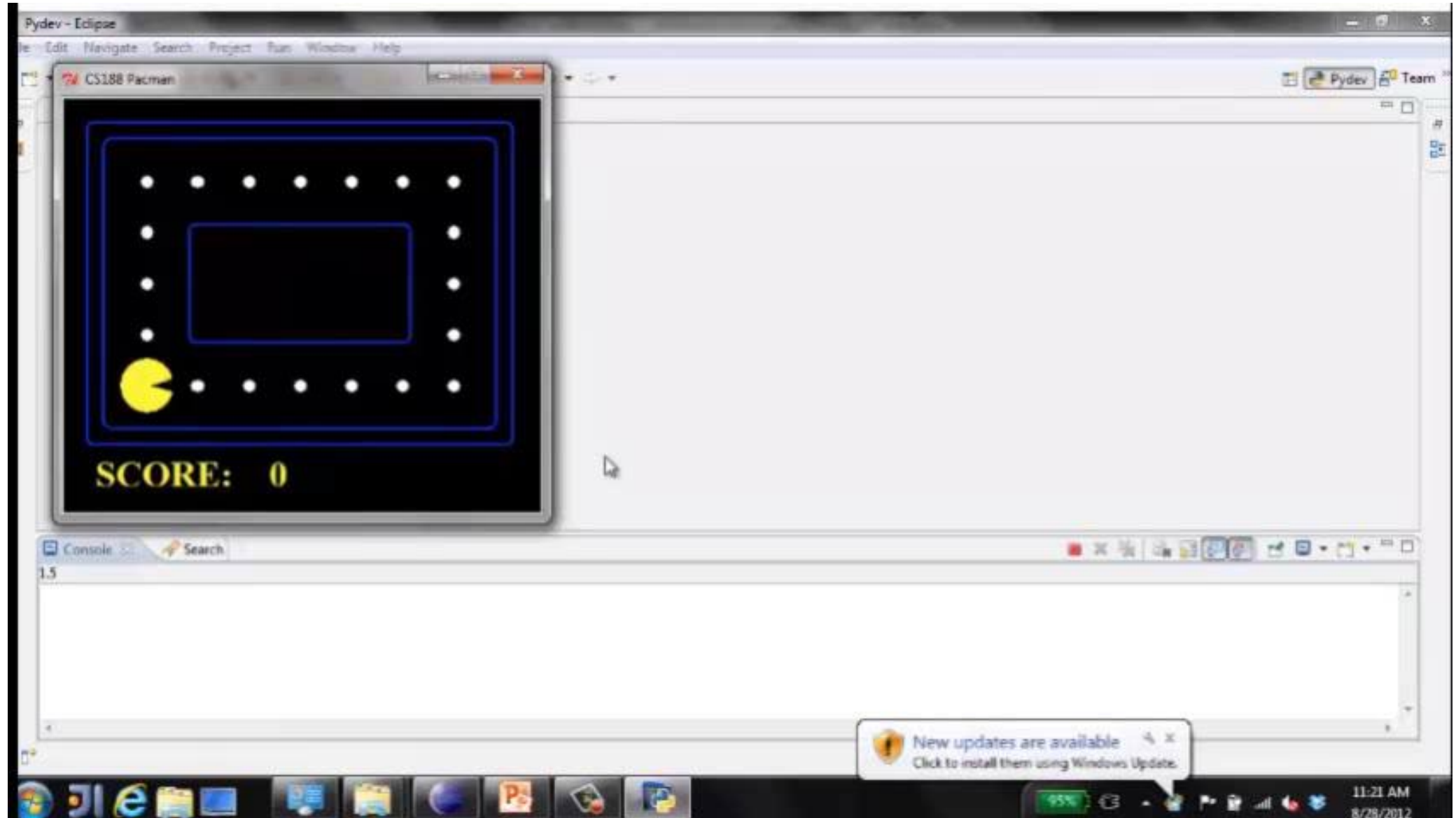
- Reflex agents:
 - Choose action based on current percept (and maybe memory)
 - May have memory or a model of the world's current state
 - Do not consider the future consequences of their actions
 - Consider how the world IS
- Can a reflex agent be rational?



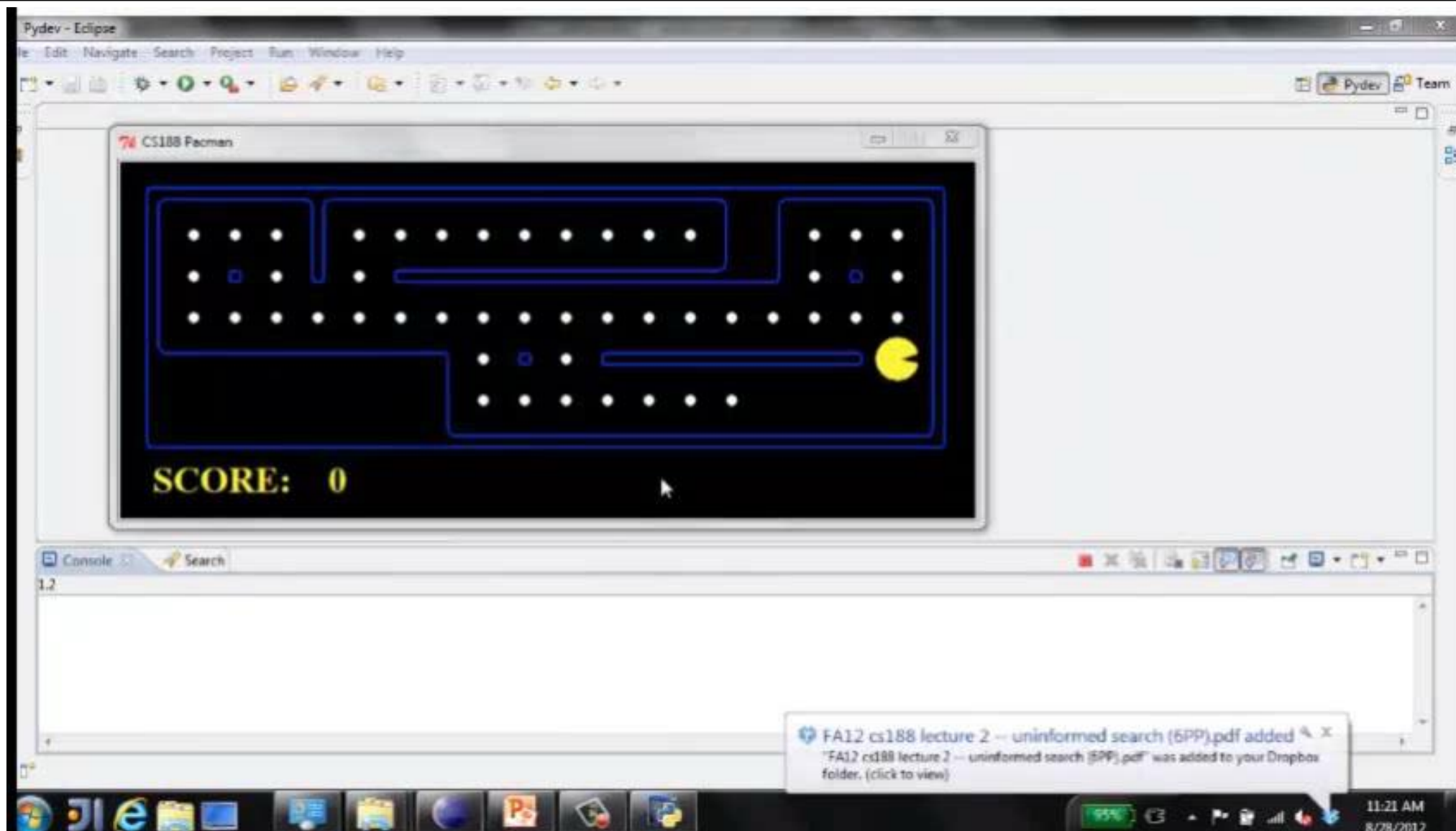
[Demo: reflex optimal (L2D1)]

[Demo: reflex optimal (L2D2)]

Video of Demo Reflex Optimal

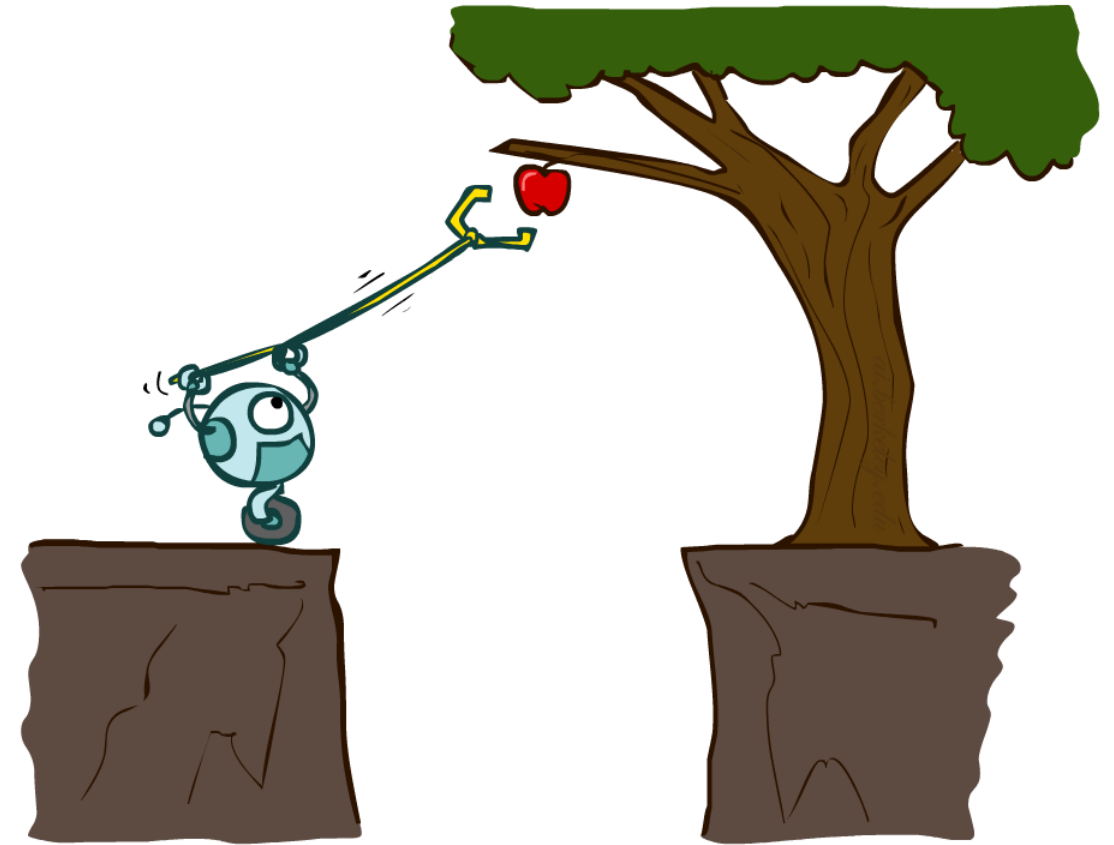


Video of Demo Reflex Odd



Planning Agents

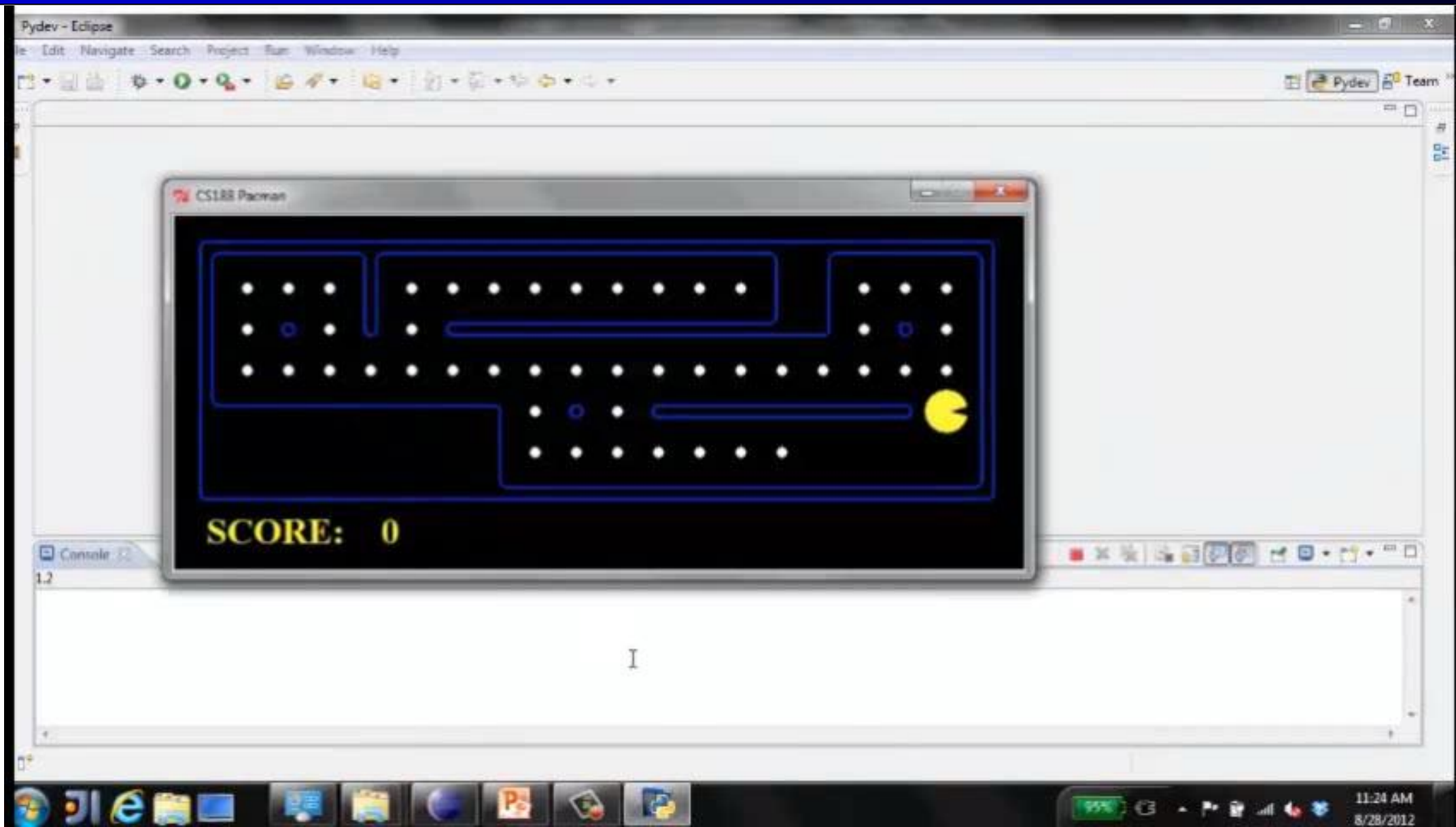
- Planning agents:
 - Ask “what if”
 - Decisions based on (hypothesized) consequences of actions
 - Must have a model of how the world evolves in response to actions
 - Must formulate a goal (test)
 - Consider how the world **WOULD BE**
- Optimal vs. complete planning
- Planning vs. replanning



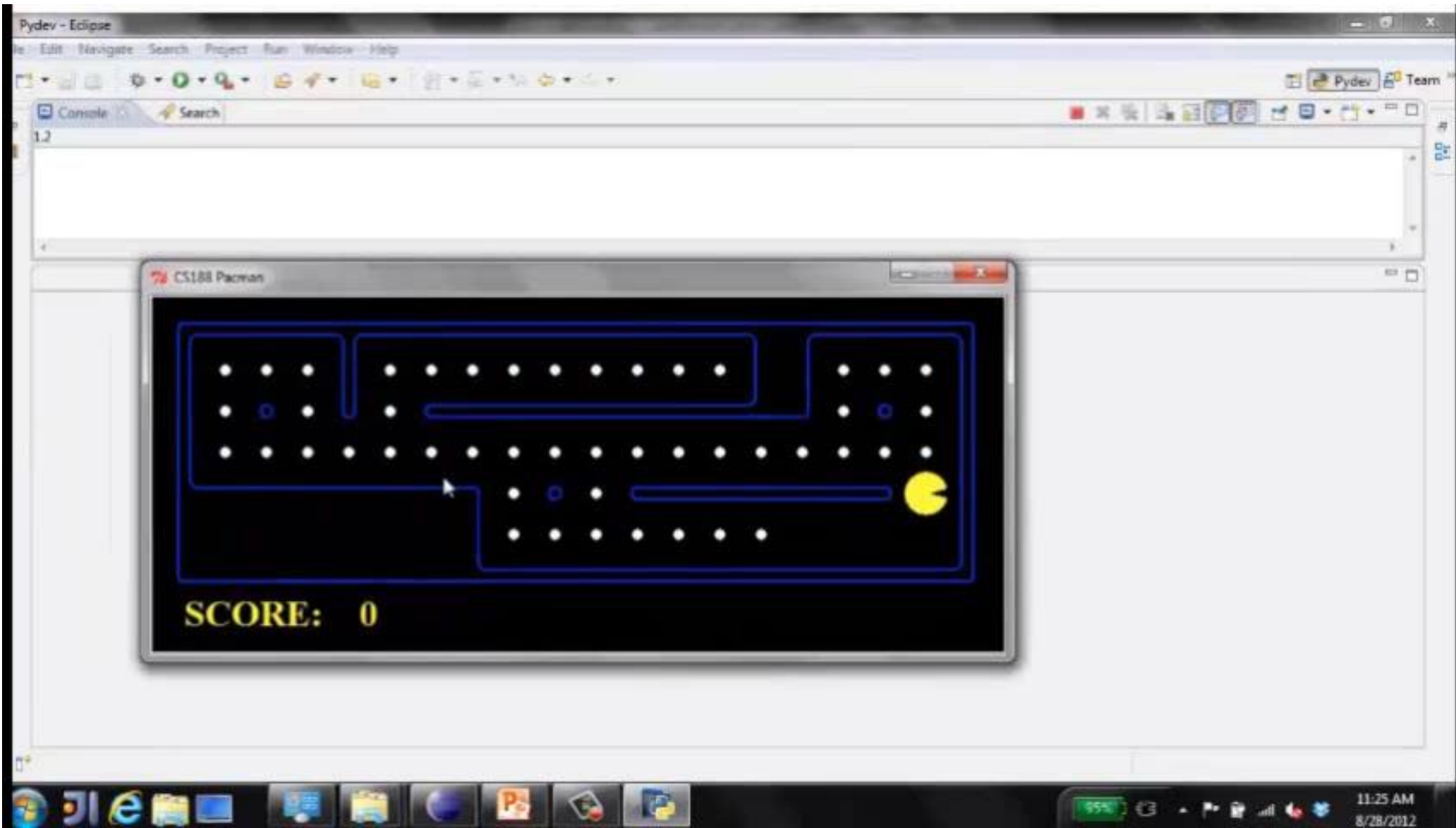
[Demo: re-planning (L2D3)]

[Demo: mastermind (L2D4)]

Video of Demo Replanning



Video of Demo Mastermind



Problem-Solving Agent

Well-defined problems

A **problem** can be defined by five components:

- **initial state, actions, transition model, goal test, path cost.**

INITIAL STATE: The **initial state** that the agent starts in.

ACTIONS: A description of the possible **actions** available to the agent.

- Given a particular state s , $ACTIONS(s)$ returns the set of actions that can be executed in s .
- Each of these actions is **applicable** in s .

TRANSITION MODEL: A description of what each action does is known as the **transition model**

- A function $RESULT(s,a)$ that returns the state that results from doing action a in state s .
- The term **successor** to refer to any state reachable from a given state by a single
- The **state space** of the problem is the set of all states reachable from the *initial state* by any sequence of actions.
- The *state space* forms a **graph** in which the nodes are states and the links between nodes are actions.
- A **path** in the state space is a sequence of states connected by a sequence of actions.

Problem-Solving Agent

Well-defined problems

GOAL TEST: The **goal test** determines whether a given state is a goal state.

PATH COST: A **path cost** function that assigns a numeric cost to each path.

- The problem-solving agent chooses a cost function that reflects its own performance measure.
 - The **cost of a path** can be described as the sum of the costs of the individual actions along the path.
 - The **step cost** of taking action a in state s to reach state s' is denoted by $c(s, a, s')$.
-
- A **SOLUTION** to a problem is an action sequence that leads from the *initial state* to a *goal state*.
 - Solution quality is measured by the path cost function, and an **OPTIMAL SOLUTION** has the lowest path cost among all solutions.

Problem Example

Travelling in Romania

- On holiday in Romania; currently in Arad.
- Flight leaves tomorrow from Bucharest

Formulate goal:

- be in Bucharest

Formulate problem:

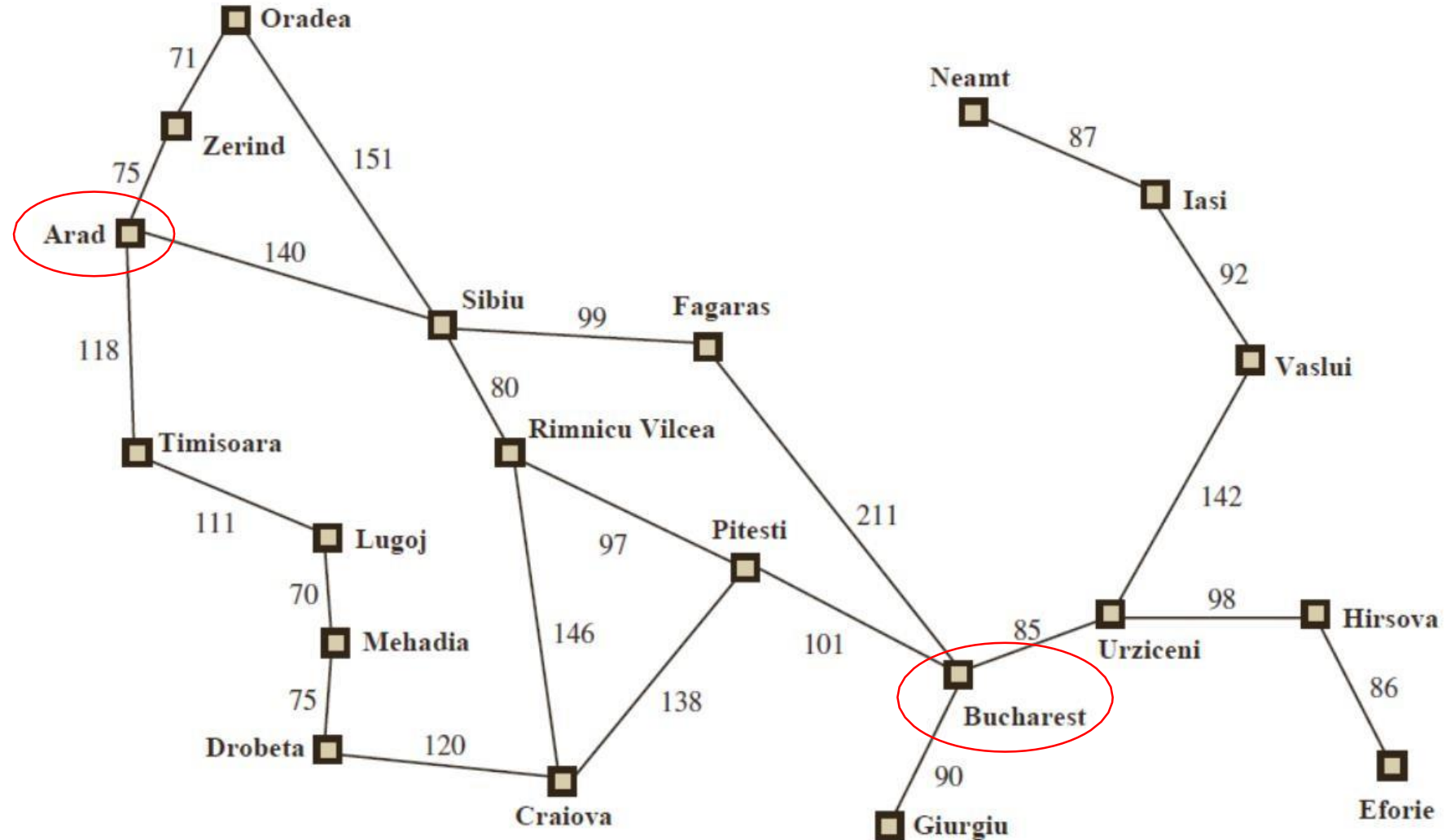
- states: various cities
- actions: drive between cities

Find solution:

- sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Problem Example

Travelling in Romania: A simplified road map



Problem Example

Travelling in Romania

- Travelling in Romania problem can be defined by:

initial state: in Arad

actions: transition model: is the graph

successor function $S(x)$ = set of action{state pairs

e.g., $S(\text{Arad}) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \langle \text{Arad} \rightarrow \text{Sibiu}, \text{Sibiu} \rangle, \langle \text{Arad} \rightarrow \text{Timisoara}, \text{Timisoara} \rangle \}$

goal test: in Bucharest

path cost: sum of distances, number of actions executed, etc.

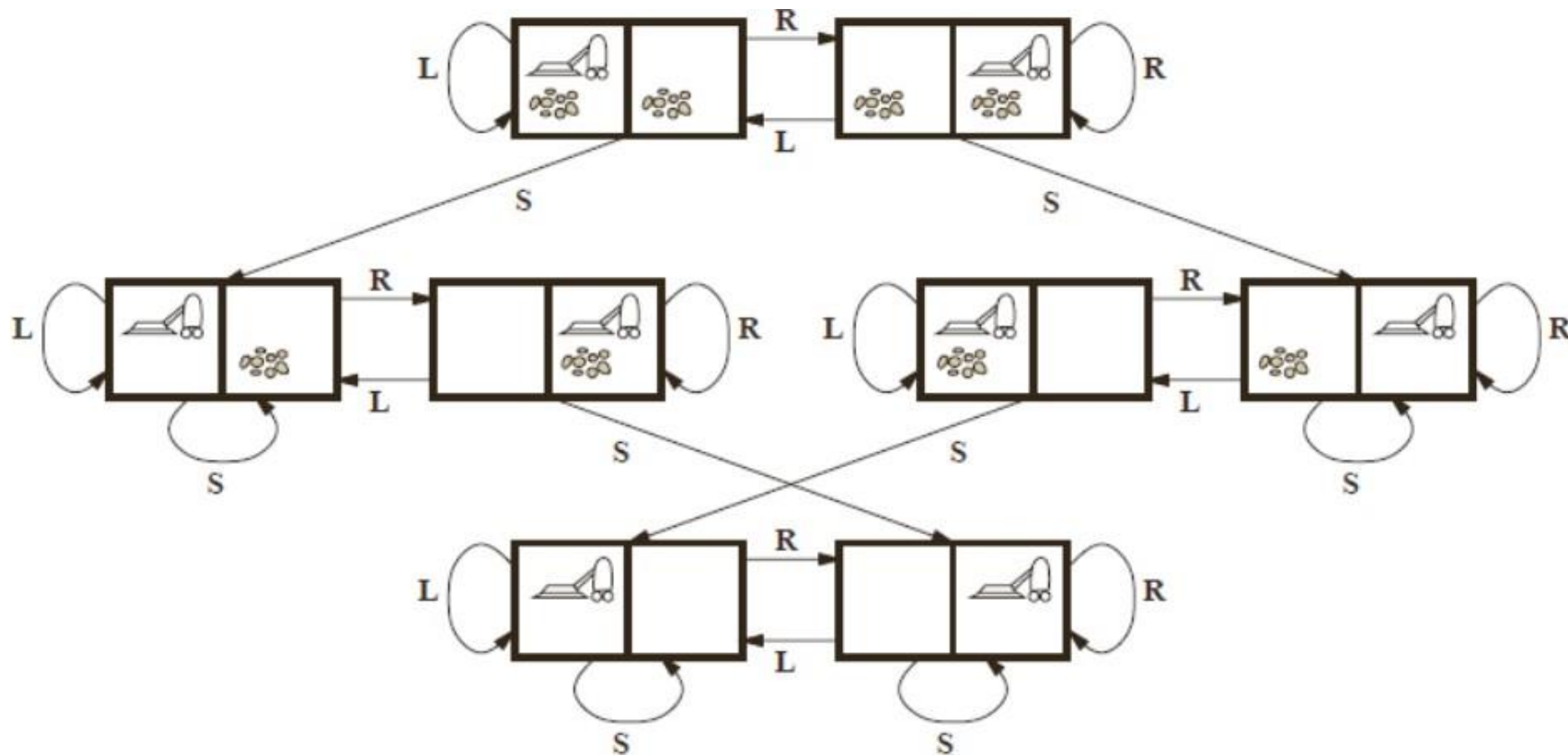
$c(x; a; y)$ is the **step cost**, assumed to be ≥ 0

- A **solution** is a sequence of actions leading from the *initial state* to a *goal state*

Problem Example

vacuum world: state space for the vacuum world

- Links denote actions: L = Left, R = Right, S = Suck.



Problem Example

vacuum world

States:

- The state is determined by both the agent location and the dirt locations.
- The agent is in one of two locations, each of which might or might not contain dirt.
- Thus, there are $2 \times 2^2 = 8$ possible world states.

Initial state: Any state can be designated as the initial state.

Actions: Each state has just three actions: *Left*, *Right*, and *Suck*.

Transition model:

- The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Sucking* in a clean square have no effect.
- The transition model defines a state space.

Goal test: This checks whether all the squares are clean.

Path cost: Each step costs 1, so the path cost is the number of steps in the path.

Problem Example

8-puzzle

- The **8-puzzle** consists of a 3×3 board with eight numbered tiles and a blank space.
- A tile adjacent to the blank space can slide into the space.
- The object is to reach a specified goal state.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Problem Example

8-puzzle

States: A state specifies the location of each of the eight tiles and the blank in one of the nine squares.

Initial state: Any state can be designated as the initial state.

- Note that goal can be reached from exactly half of the possible initial states.

Actions: Movements of the blank space *Left, Right, Up, or Down*.

- Different subsets of these are possible depending on where the blank is.

Transition model: Given a state and action, this returns the resulting state;

Goal test: This checks whether the state matches the goal configuration

Path Cost: Each step costs 1, so the path cost is the number of steps in the path.

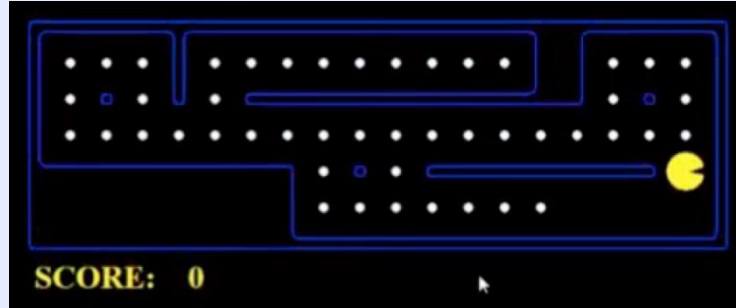
Problem Example

8-puzzle

- The 8-puzzle belongs to the family of **sliding-block puzzles**,
- This family is known to be **NP-complete**.
 - Optimal solution of n-Puzzle family is NP-hard. ie NO polynomial solution for the problem.
- The 8-puzzle has $9!/2=181,440$ reachable states.
- The 15-puzzle (on a 4×4 board) has around 1.3 trillion states,
- The 24-puzzle (on a 5×5 board) has around 10^{25} states

What's in a State Space?

The **world state** includes every last detail of the environment



A **search state** keeps only the details needed for planning (abstraction)

■ Problem: Pathing

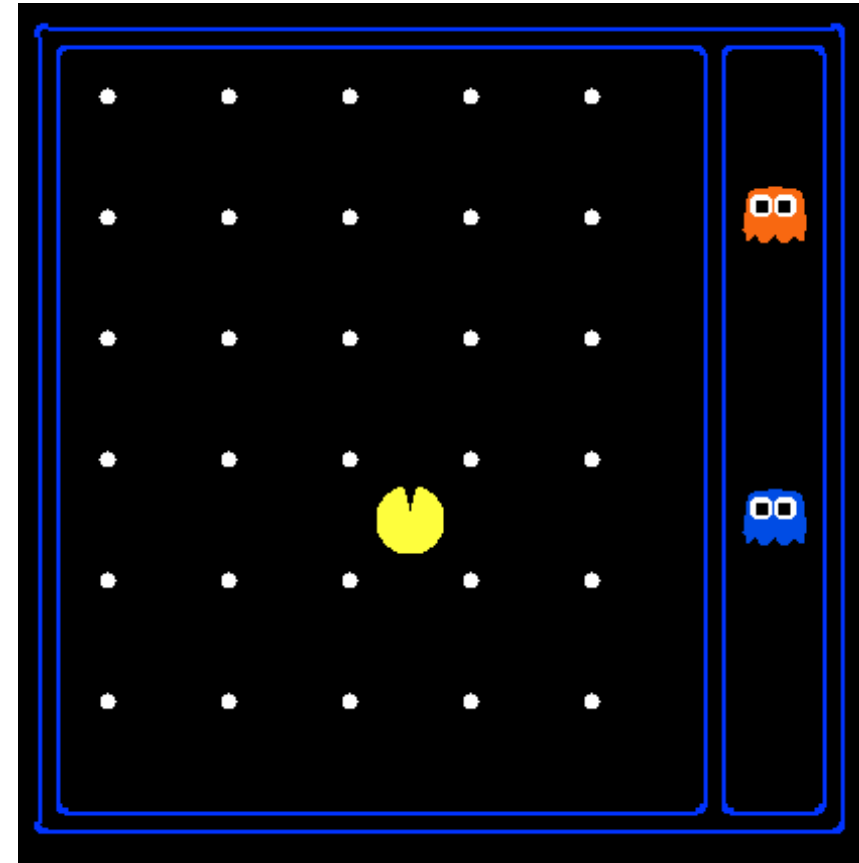
- States: (x,y) location
- Actions: NSEW
- Successor: update location only
- Goal test: is $(x,y)=\text{END}$

■ Problem: Eat-All-Dots

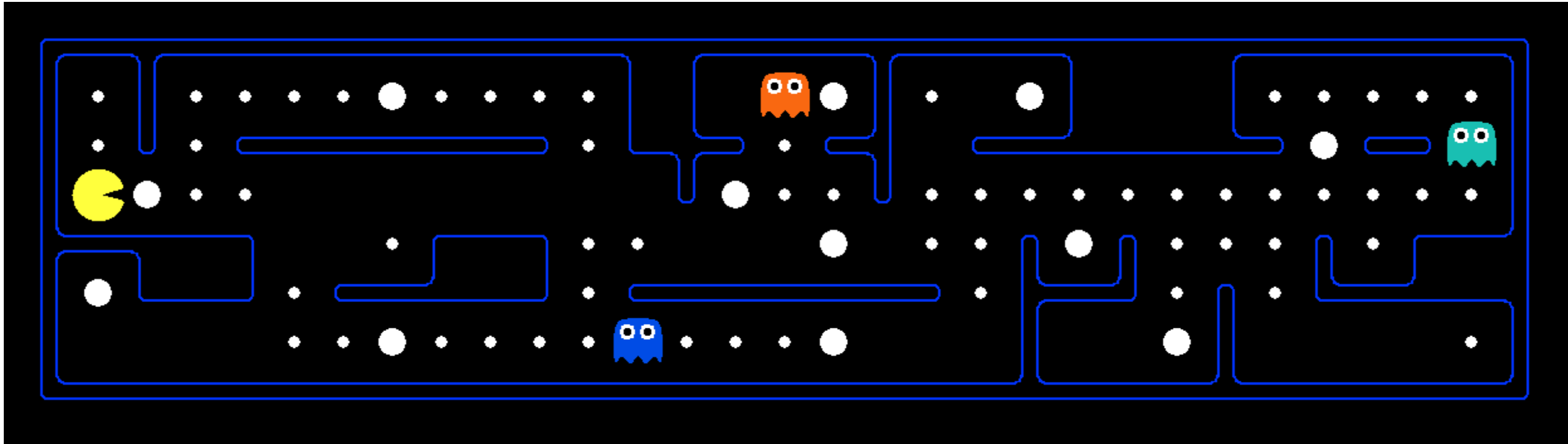
- States: $\{(x,y), \text{dot booleans}\}$
- Actions: NSEW
- Successor: update location and possibly a dot boolean
- Goal test: dots all false

State Space Sizes?

- World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many
 - World states?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - States for pathing?
120
 - States for eat-all-dots?
 $120 \times (2^{30})$



Quiz: Safe Passage



- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
 - (agent position, dot booleans, power pellet booleans, remaining scared time)

Real-World Problems

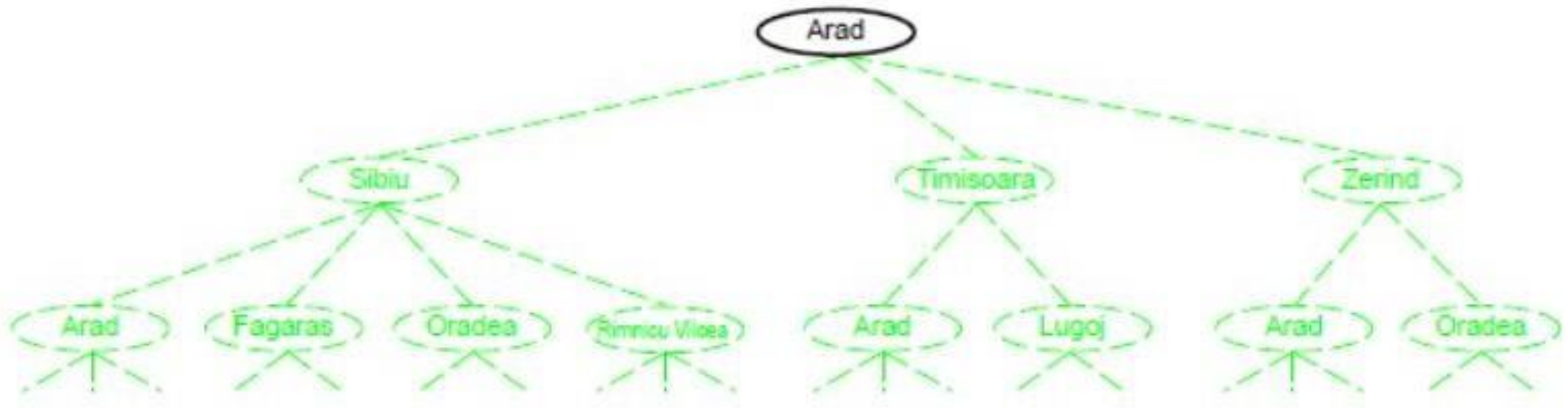
- **Route-finding problem** is defined in terms of specified locations and transitions along links between them.
- Route-finding algorithms are used in a variety of applications such as Web sites and in-car systems that provide driving directions.
- **VLSI layout problem** requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield.

Searching for Solutions

- A solution is an action sequence and **search algorithms** considers various possible action sequences.
- The possible action sequences starting at the initial state form a **search tree** with the initial state at the root;
- The **branches** are **actions** and the **nodes** correspond to **states** in the state space of the problem.
- **Expanding** the current state is application of each legal action to the current state and generation of a new set of states.
 - The current state is the **parent node**, newly generated states are **child nodes**
- **Leaf node** is a node with no children in the tree.
- The *set of all leaf nodes available* for expansion at any given point is called the **frontier**.
- **Search algorithms** all share this basic structure; they vary primarily according to how they choose which state to expand next: **search strategy**.

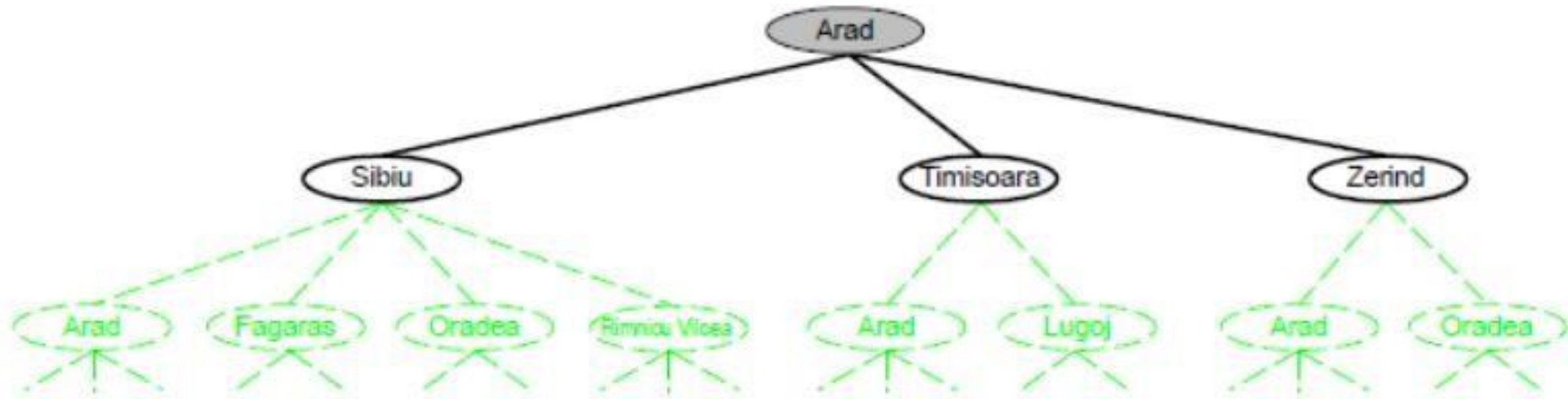
Partial Search Trees for Travelling in Romania

initial state



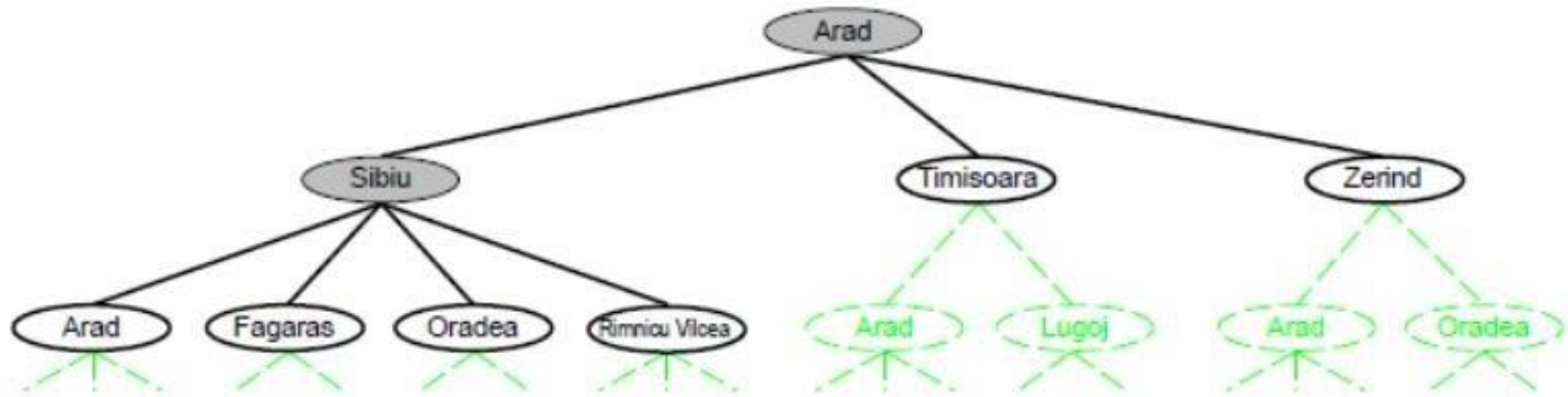
Partial Search Trees for Travelling in Romania

After expanding Arad



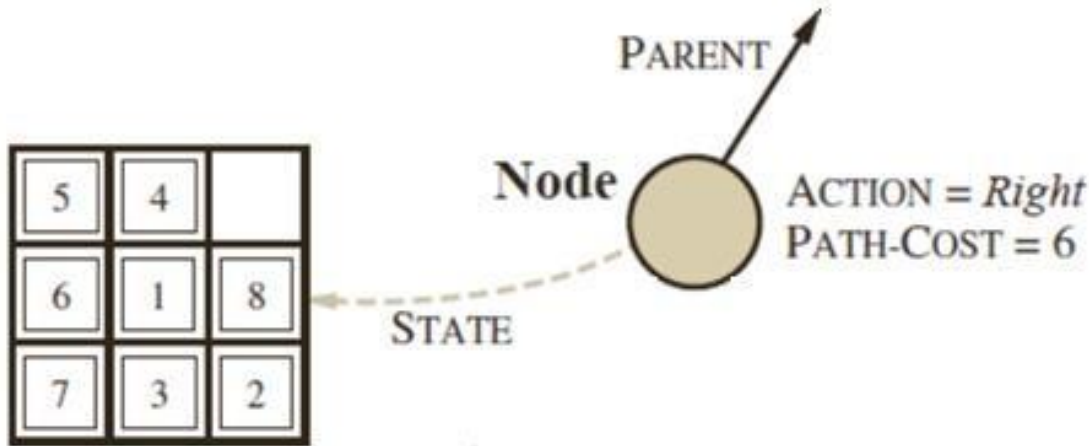
Partial Search Trees for Travelling in Romania

After expanding Sibiu



Infrastructure for Search Algorithms

- Search algorithms require a data structure to keep track of the search tree that is being constructed.
- Each node n of the tree contains four components:
 - n .STATE: the state in the state space to which the node corresponds;
 - n .PARENT: the node in the search tree that generated this node;
 - n .ACTION: the action that was applied to the parent to generate the node;
 - n .PATH-COST: the cost of the path from the initial state to the node,



Informal Description of Graph Search Algorithms

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure

- initialize *frontier* using *initial state* of *problem*
- initialize *explored set* to be empty
- **loop do**
 - **if** *frontier* is empty **then return** *failure*
 - choose a *leaf node* from *frontier* and remove it from there
 - **if** *node* contains a *goal state* **then return** corresponding *solution*
 - add *node* to *explored set*
 - expand *node*, adding *resulting nodes* to *frontier* only if not in *frontier* or *explored set*

Informal Description of Graph Search Algorithms

- To avoid exploring *redundant paths* is to remember them.
- **Explored set** (also known as *closed list*) remembers every expanded node.
- The **frontier** needs to be stored in such a way that *search algorithm* can easily choose next node to expand according to its *preferred strategy*.
 - The appropriate data structure for this is a *queue*.
- **Queues** are characterized by the order in which they store the inserted nodes.
 - First-in, first-out or FIFO queue, which pops the oldest element of the queue; (**QUEUE**)
 - Last-in, first-out or LIFO queue (also known as **STACK**), which pops the newest element
 - **PRIORITY QUEUE**, which pops the element of the queue with the highest priority according to some ordering function.
- The *explored set* can be implemented with a hash table to allow efficient checking for repeated states.

Measuring Problem-Solving Performance

- We can evaluate a **search algorithm's performance** in four ways:

Completeness: Is the algorithm guaranteed to find a solution when there is one?

Optimality: Does the strategy find the optimal solution?

Time complexity: How long does it take to find a solution?

Space complexity: How much memory is needed to perform the search?

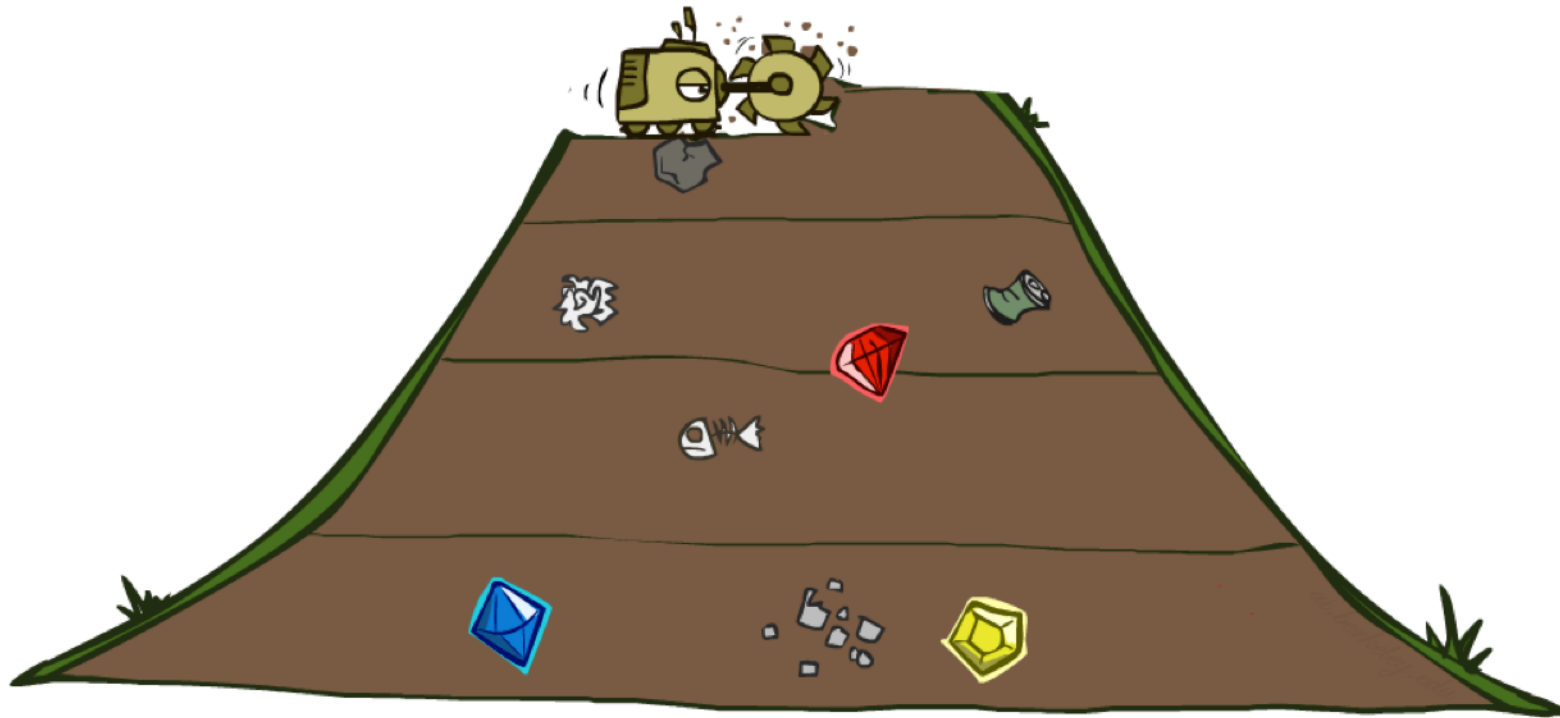
Measuring Problem-Solving Performance

- The typical measure for *time and space complexity* is the size of the state space graph, $|V| + |E|$, where V is the set of vertices (nodes) of the graph and E is the set of edges (links).
 - This is appropriate when the graph is an explicit data structure.
- In AI, graph is often represented implicitly by the initial state, actions, and transition model and is frequently infinite. For these reasons, complexity is expressed in terms of three quantities:
 - **b - maximum branching factor of the search tree**
 - **d - depth of the least-cost solution**
 - **m - maximum depth of the state space (may be ∞)**
- Time is often measured in terms of the number of nodes generated during the search, and space in terms of the maximum number of nodes stored in memory.

Uninformed Search Strategies

- **Uninformed strategies** use only the information available in the problem definition.
- All they can do is generate successors and distinguish a goal state from a non-goal state.
- All search strategies are distinguished by the order in which nodes are expanded.
- Uninformed (blind) Search Strategies are:
 - **Breadth-first Search**
 - **Depth-first Search**
 - **Depth-limited Search**
 - **Iterative Deepening Search**
 - **Uniform-cost Search**

Breadth-First Search



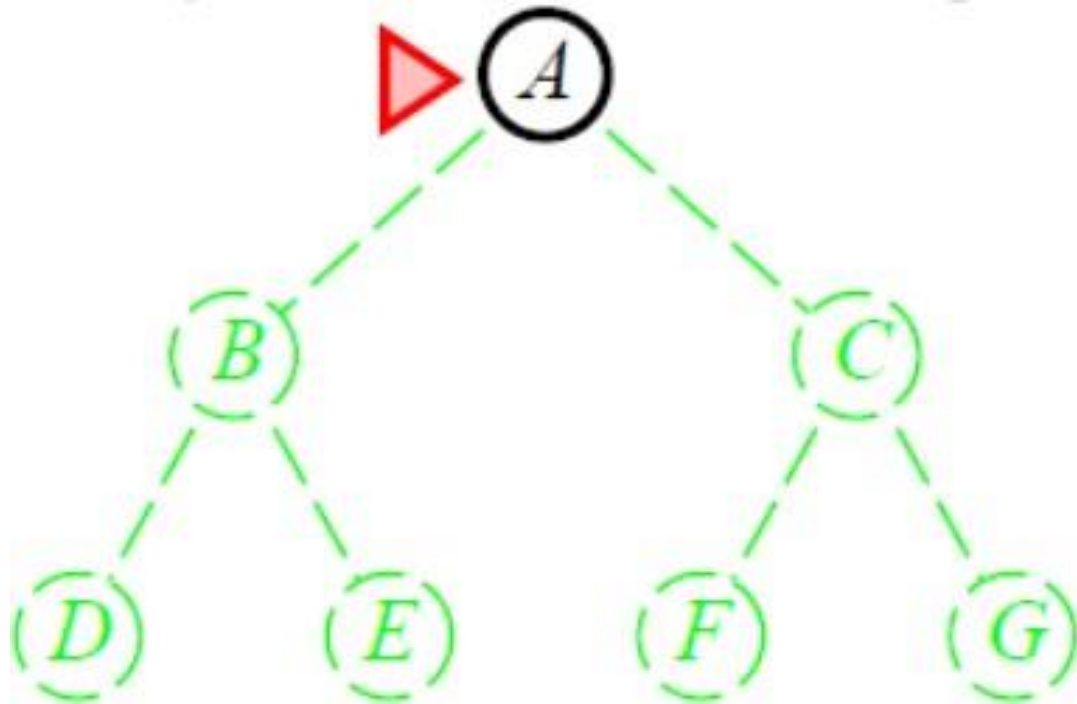
Breadth-First Search

- **Breadth-first search** is a simple strategy in which the root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on.
 - All nodes are expanded at a given depth in search tree before any nodes at next level are expanded.
- **Breadth-first search** uses a **FIFO** queue for the *frontier*

Breadth-First Search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure  
  node  $\leftarrow$  a node with STATE = problem.INITIAL-STATE, PATH-COST = 0  
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)  
  frontier  $\leftarrow$  a FIFO queue with node as the only element  
  explored  $\leftarrow$  an empty set  
  loop do  
    if EMPTY?(frontier) then return failure  
    node  $\leftarrow$  POP(frontier) /* chooses the shallowest node in frontier */  
    add node.STATE to explored  
    for each action in problem.ACTIONS(node.STATE) do  
      child  $\leftarrow$  CHILD-NODE(problem, node, action)  
      if child.STATE is not in explored or frontier then  
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)  
        frontier  $\leftarrow$  INSERT(child, frontier)
```

Breadth-First Search: Example



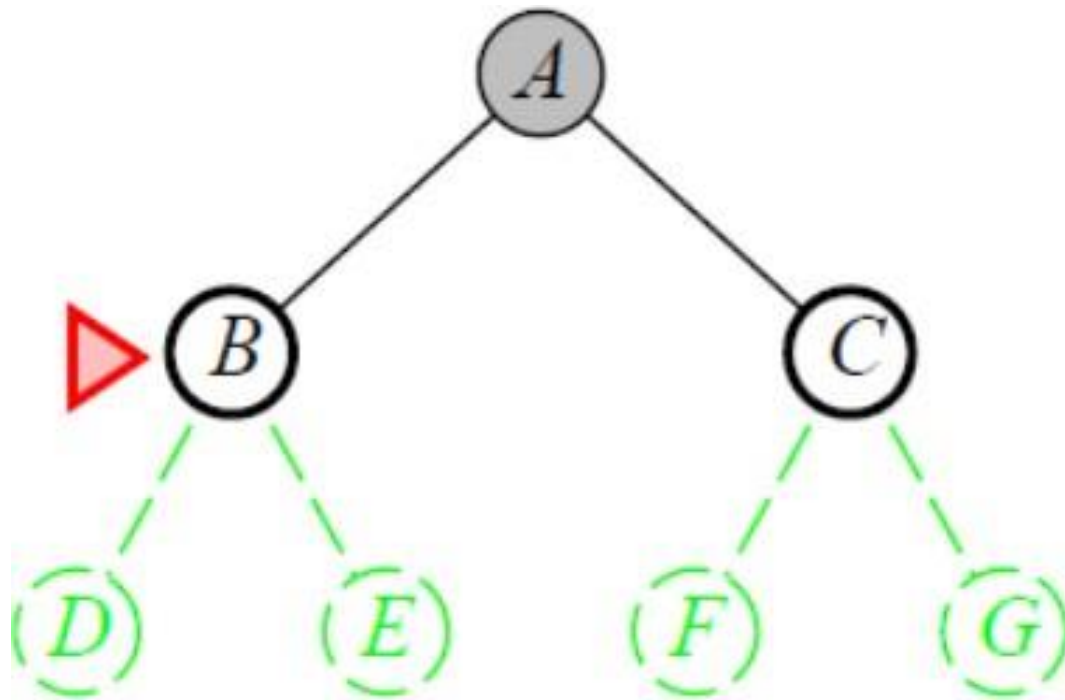
Frontier

A

Explored

empty

Breadth-First Search: Example



Frontier

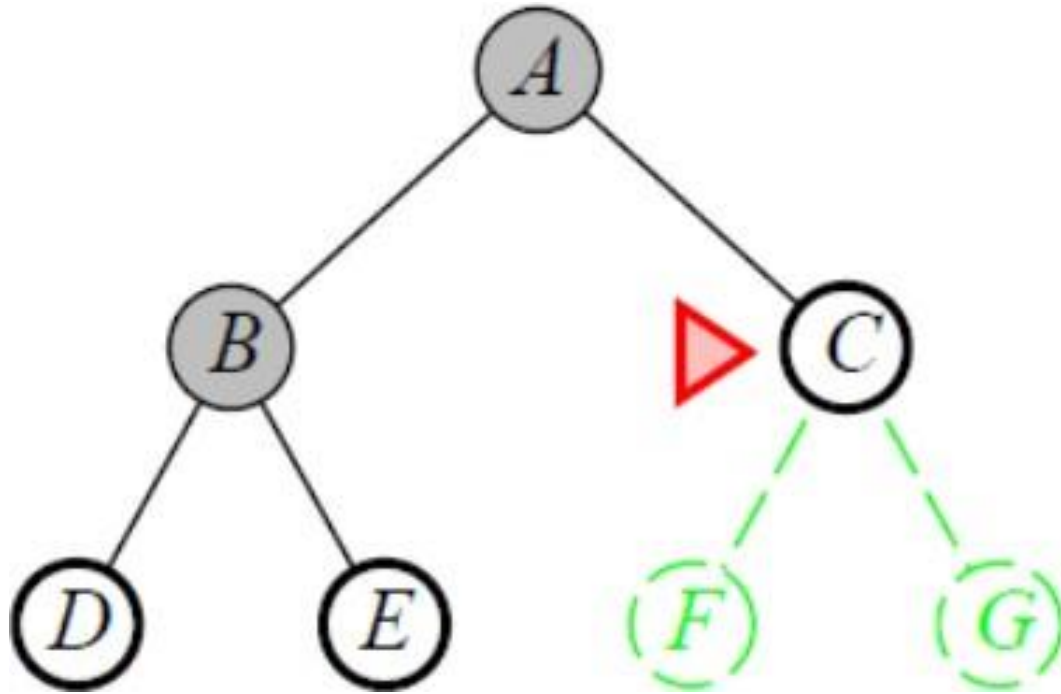
B

C

Explored

A

Breadth-First Search: Example



Frontier

C

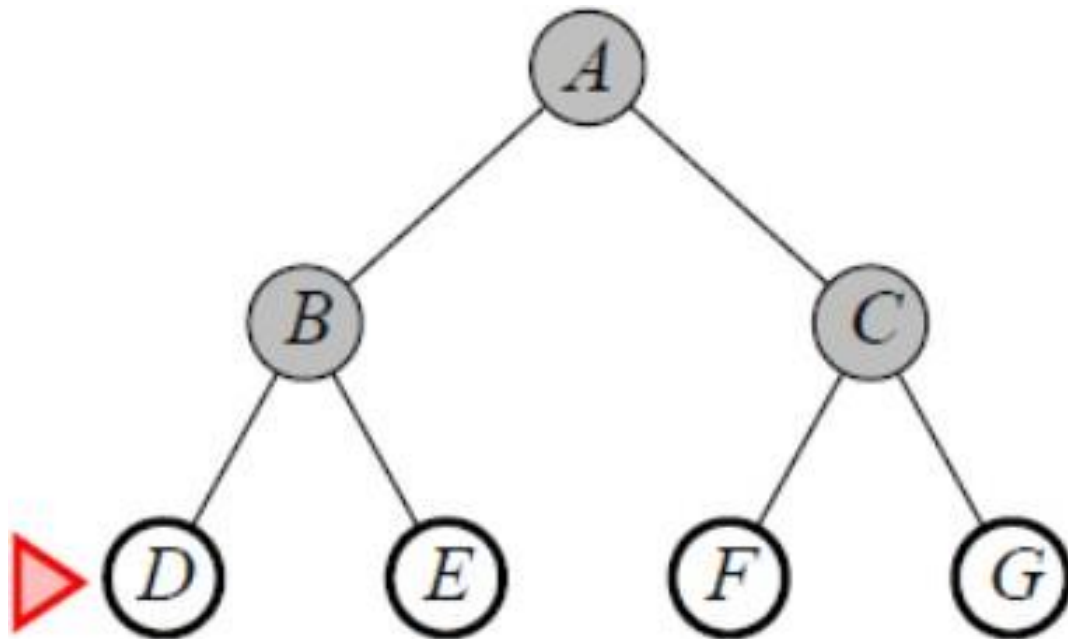
D

E

Explored

A, B

Breadth-First Search: Example



Frontier

D

E

F

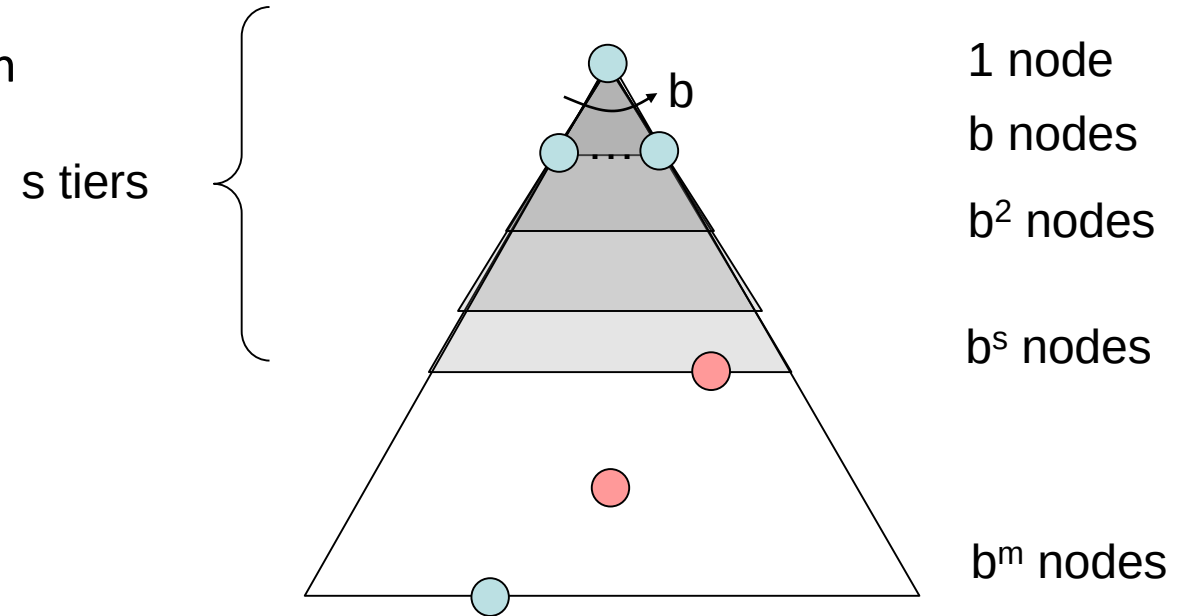
G

Explored

A, B, C

Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be s
 - Search takes time $O(b^s)$
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^s)$
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)



Breadth-First Search

- Time and memory requirements for breadth-first search assuming branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

- The memory requirements are a bigger problem for breadth-first search than is the execution time.

Depth-First Search

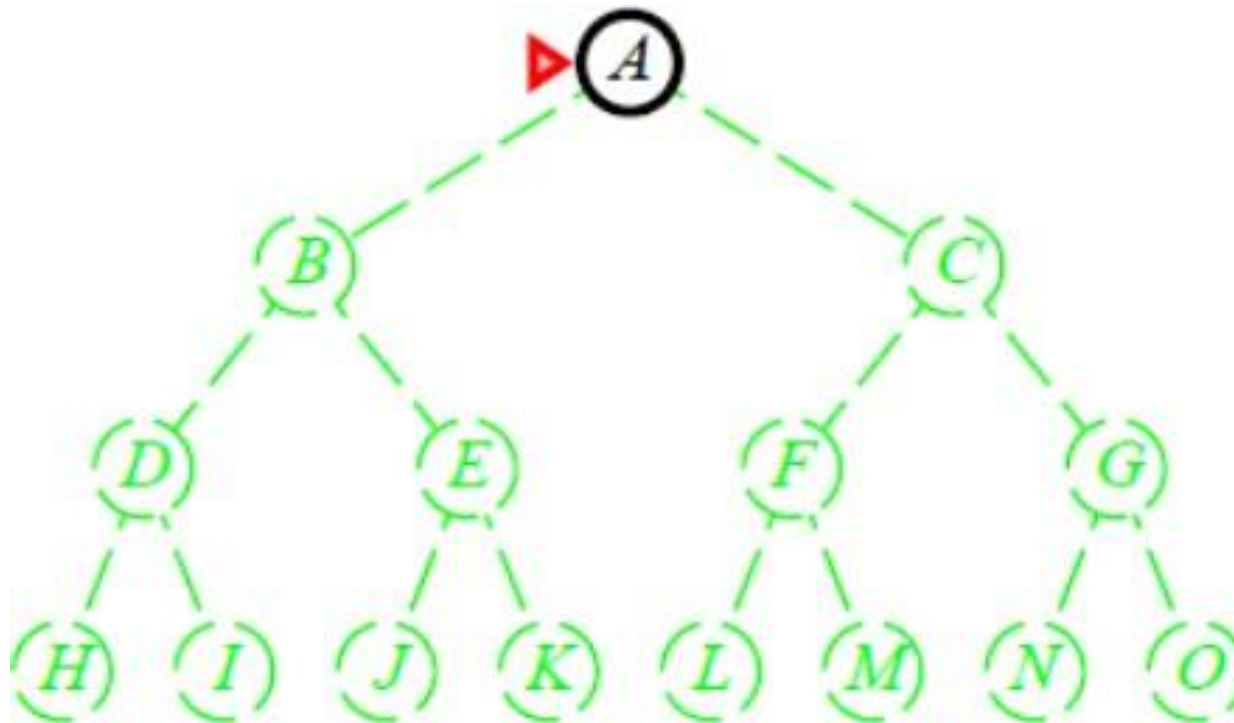


Depth-First Search

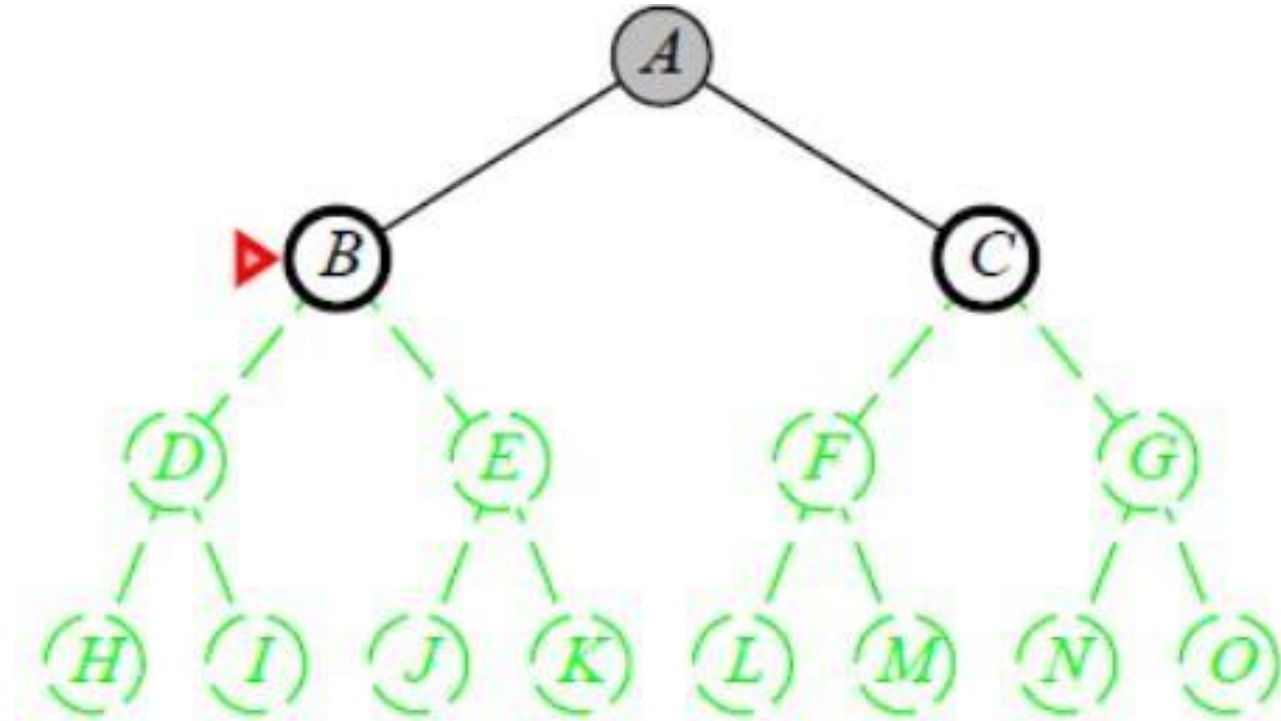
- **Depth-first search** always expands *deepest unexpanded node* in *frontier* of search tree.
 - As nodes are expanded, they are dropped from frontier, so then search “backs up” to next deepest node that still has unexplored successors.
- **Depth-first search** uses a **LIFO queue (STACK)**.
 - A LIFO queue means that the most recently generated node is chosen for expansion.
 - This must be the deepest unexpanded node because it is one deeper than its parent.

Depth-First Search

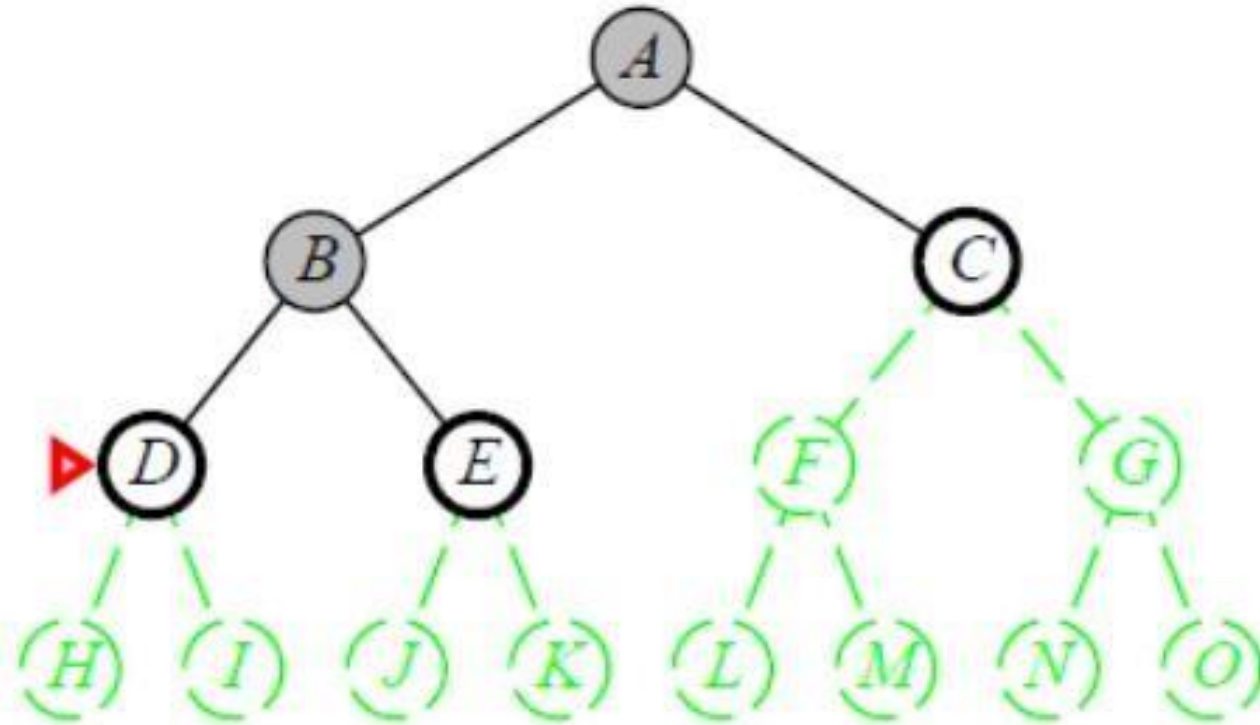
- Depth-first search on a binary tree.
- Explored nodes with no descendants in the frontier are removed from memory.
- Nodes at depth 3 have no successors and M is the only goal node.



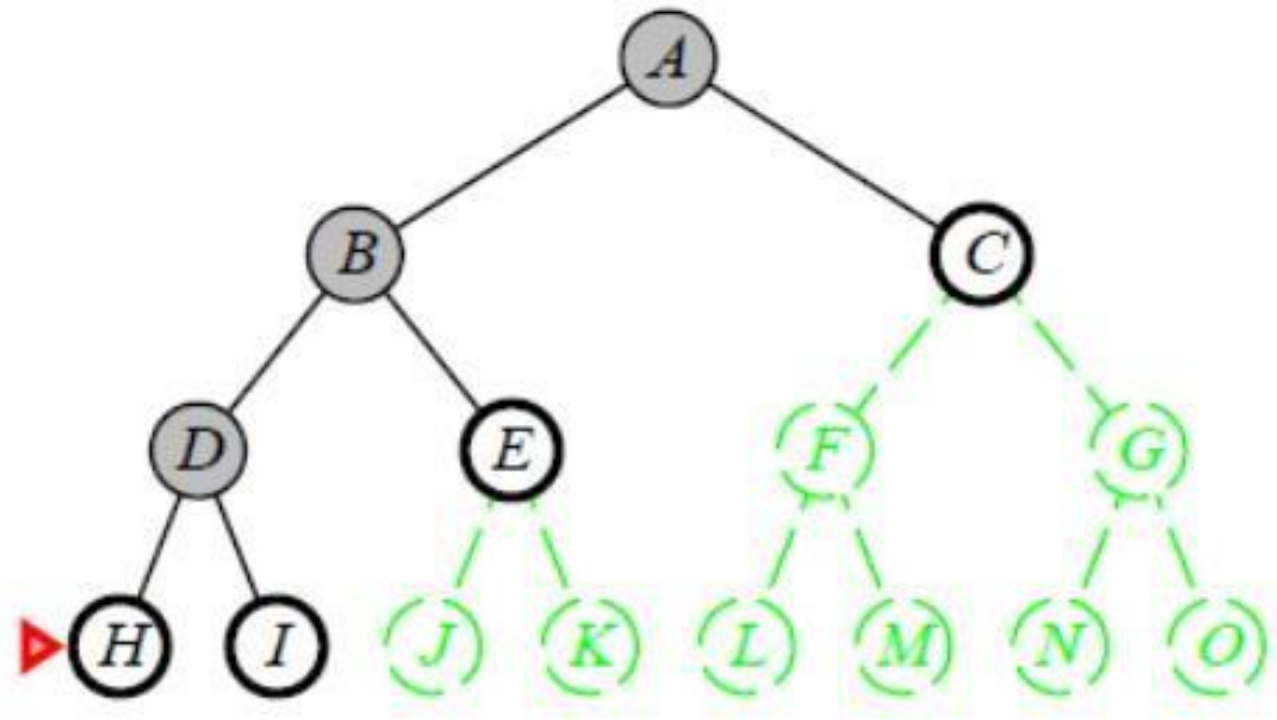
Depth-First Search



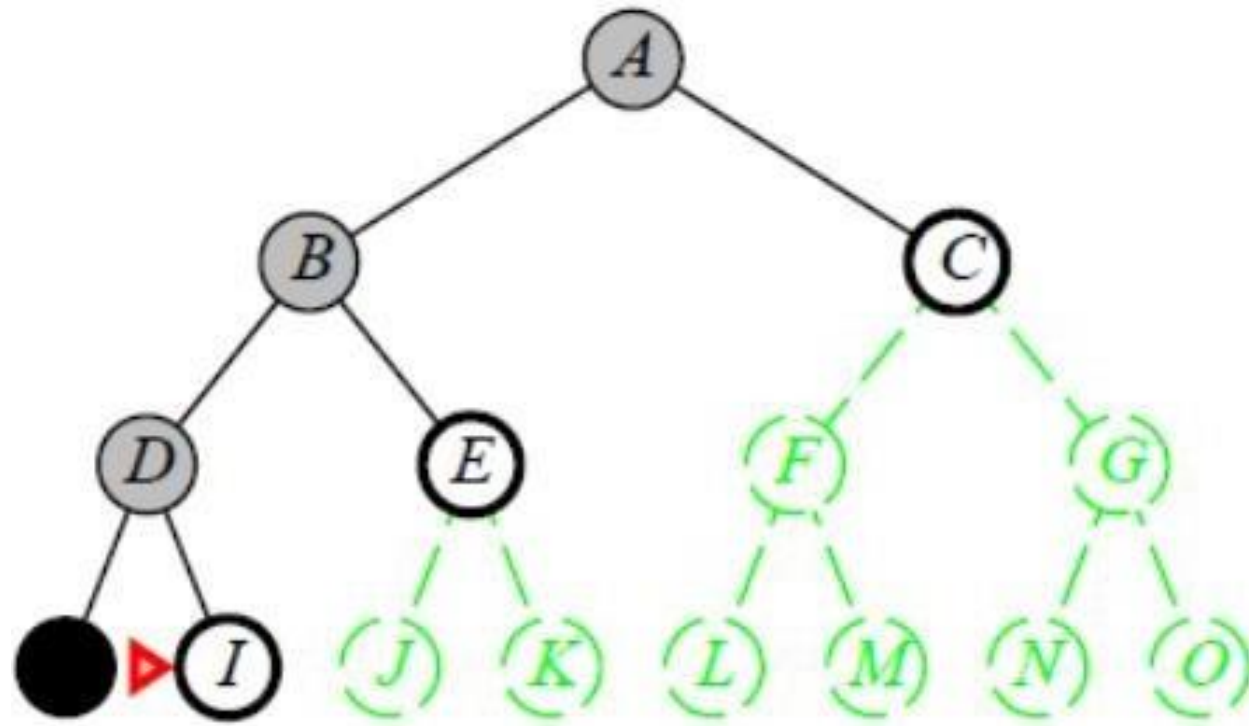
Depth-First Search



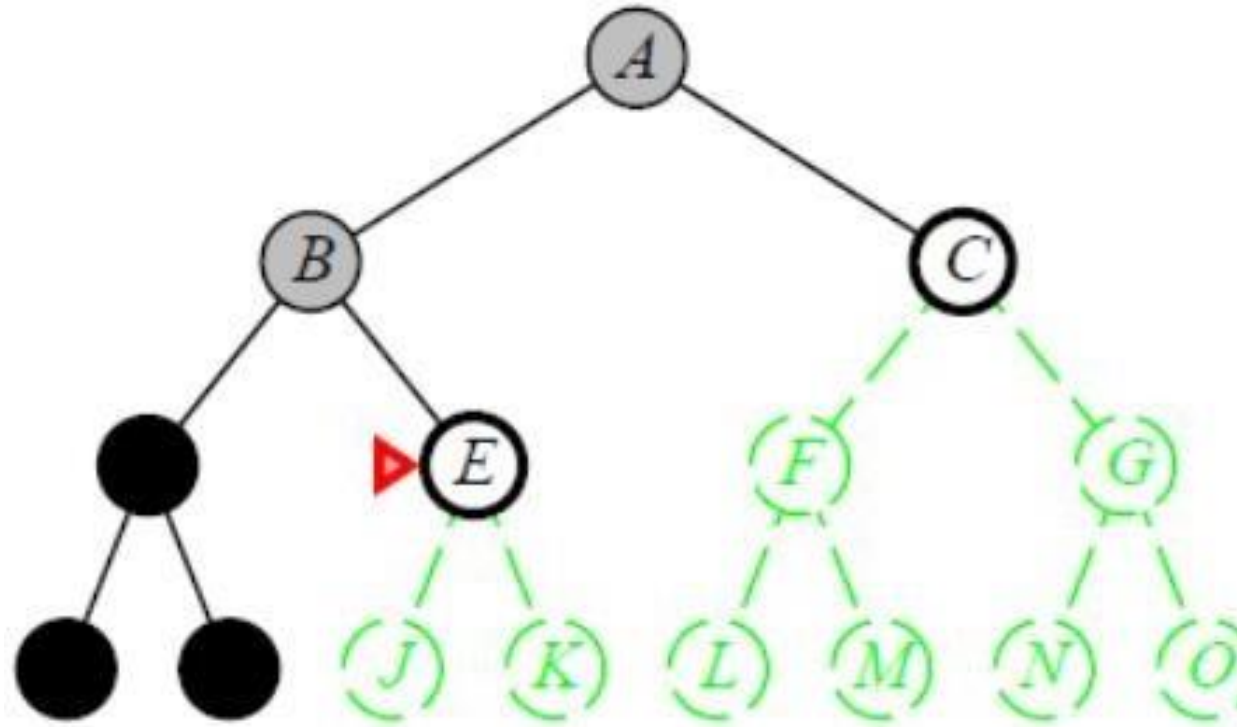
Depth-First Search



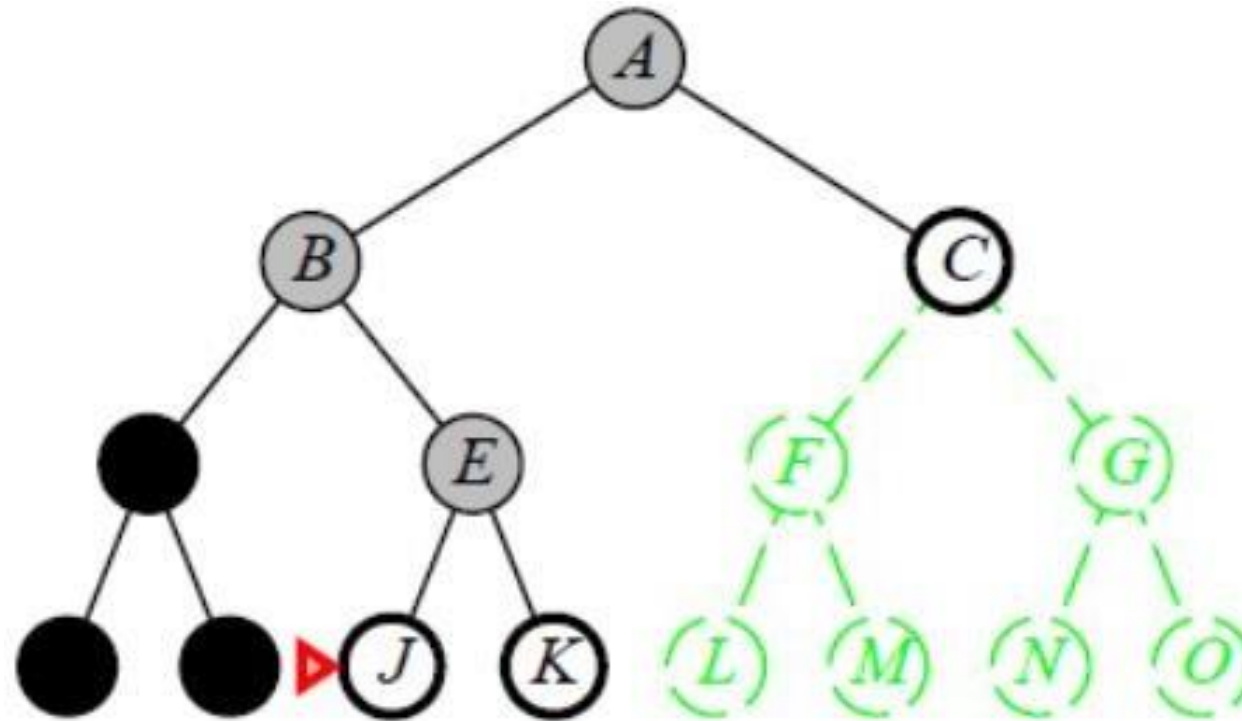
Depth-First Search



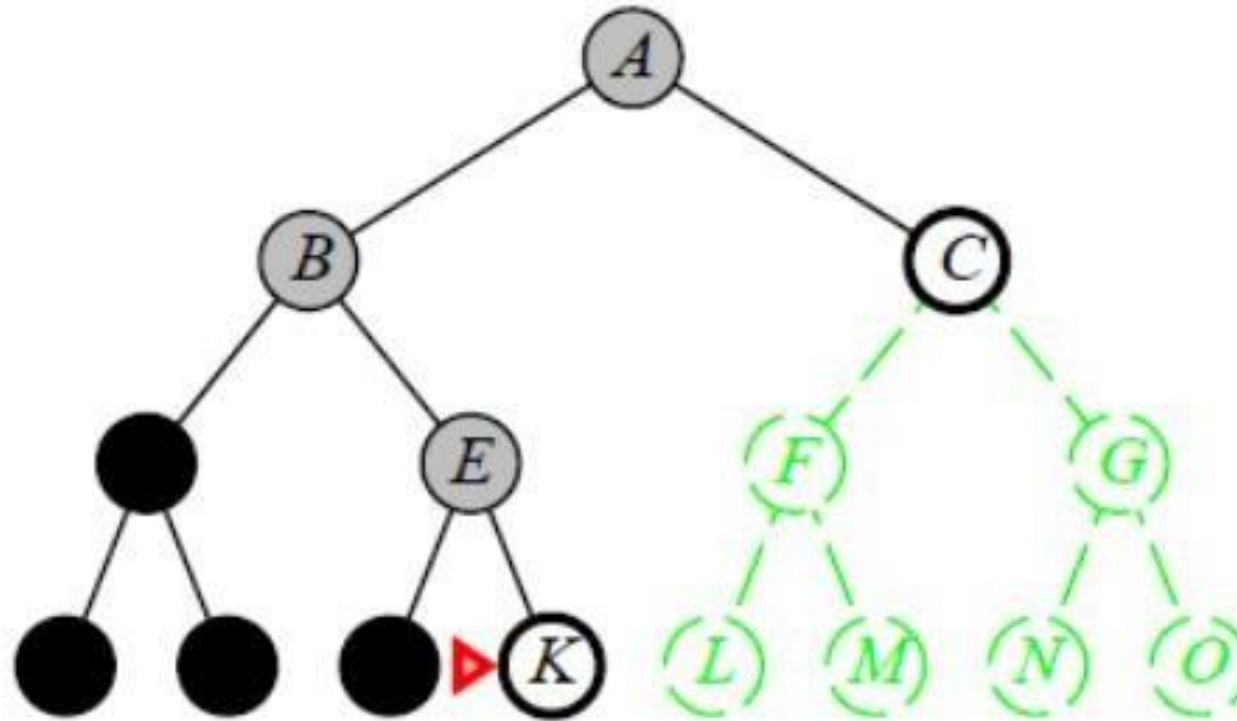
Depth-First Search



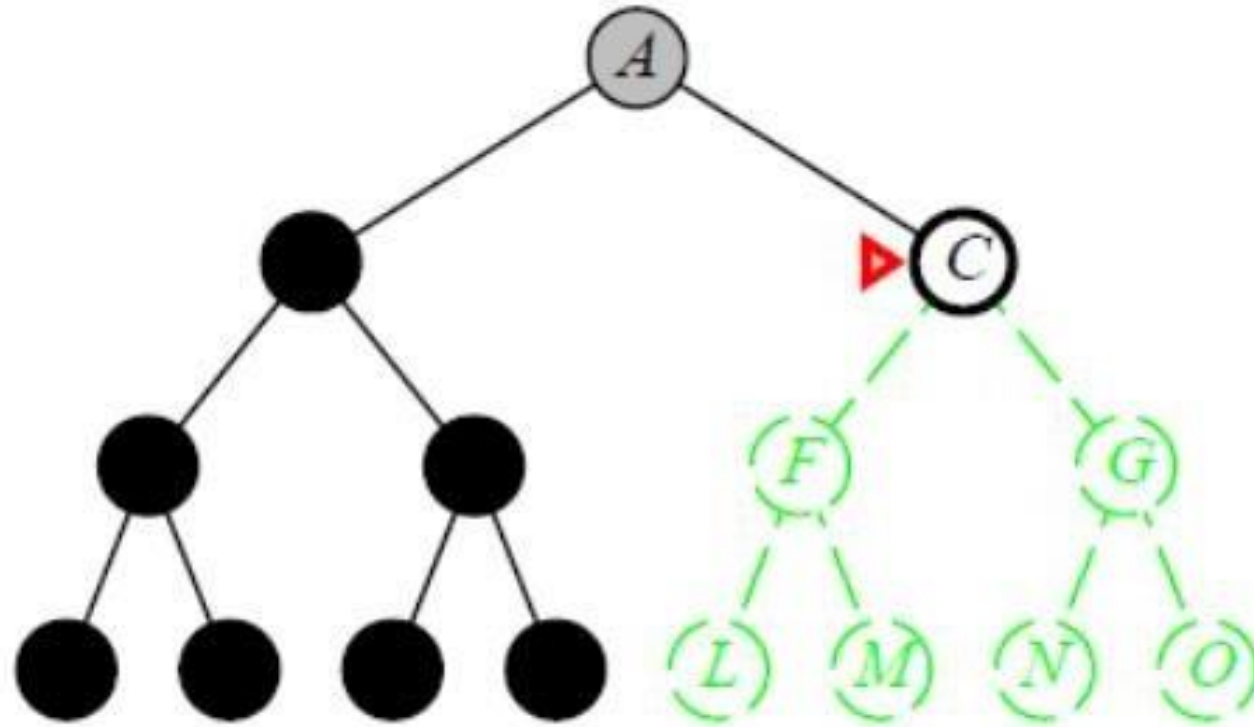
Depth-First Search



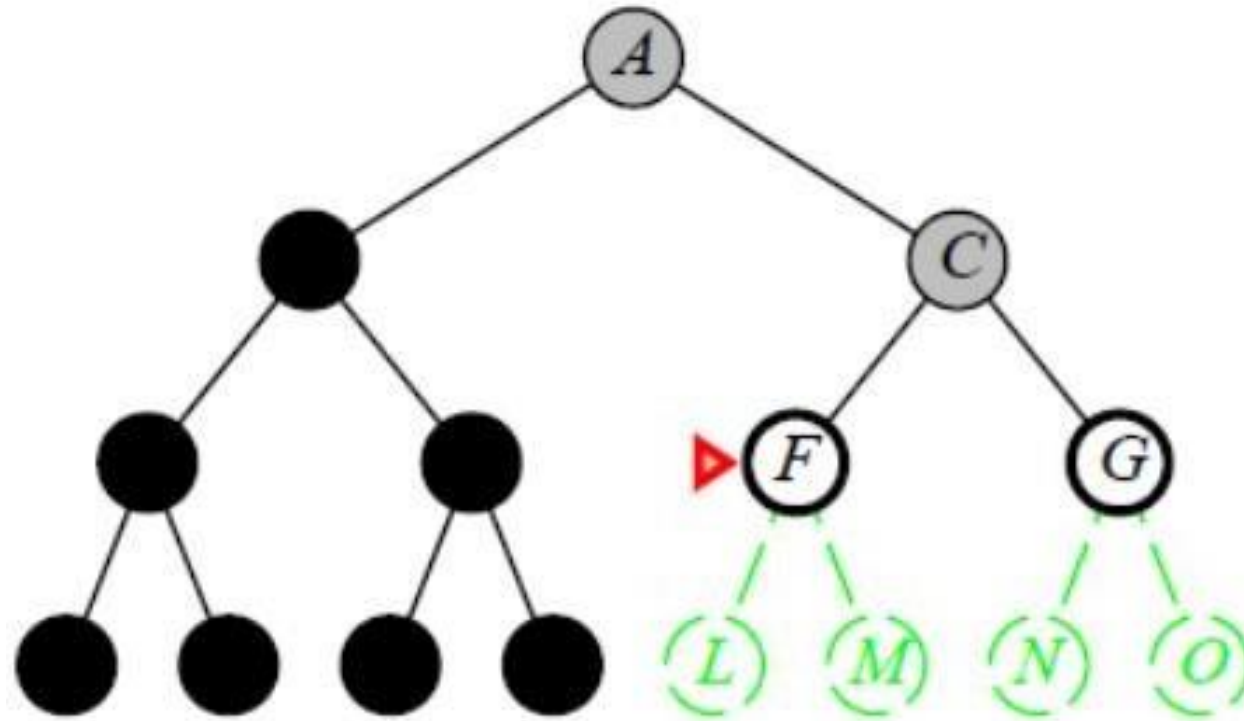
Depth-First Search



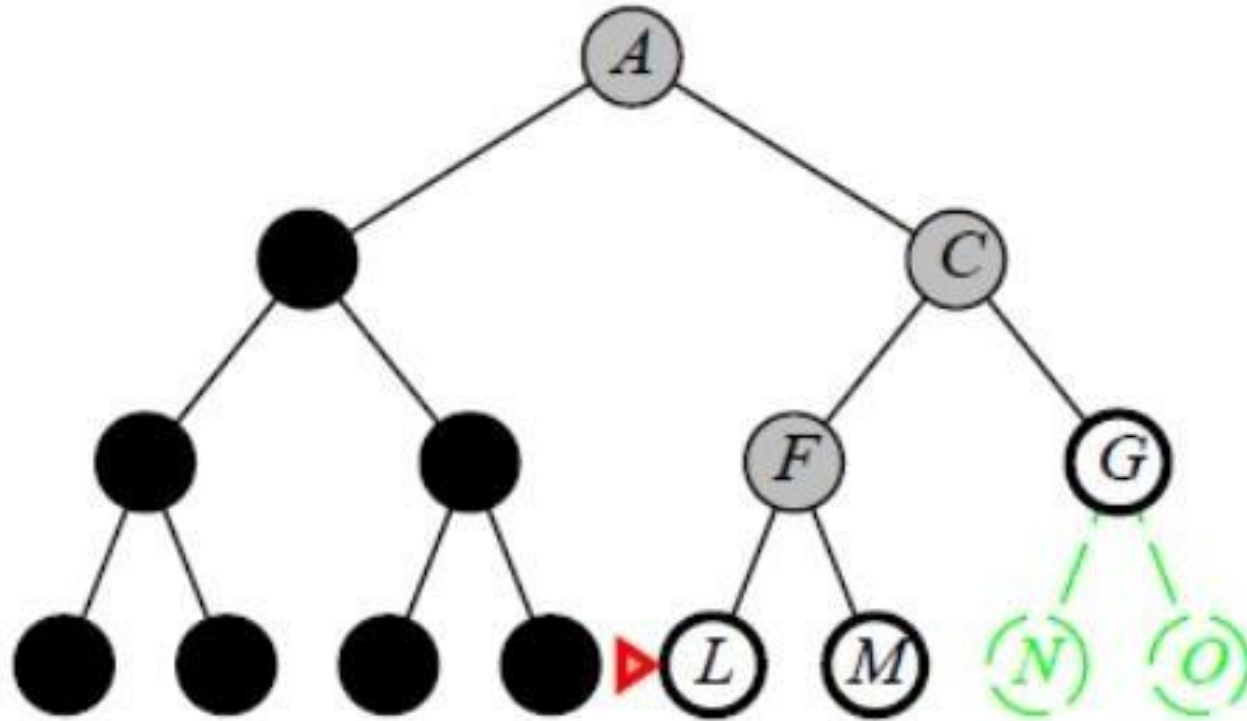
Depth-First Search



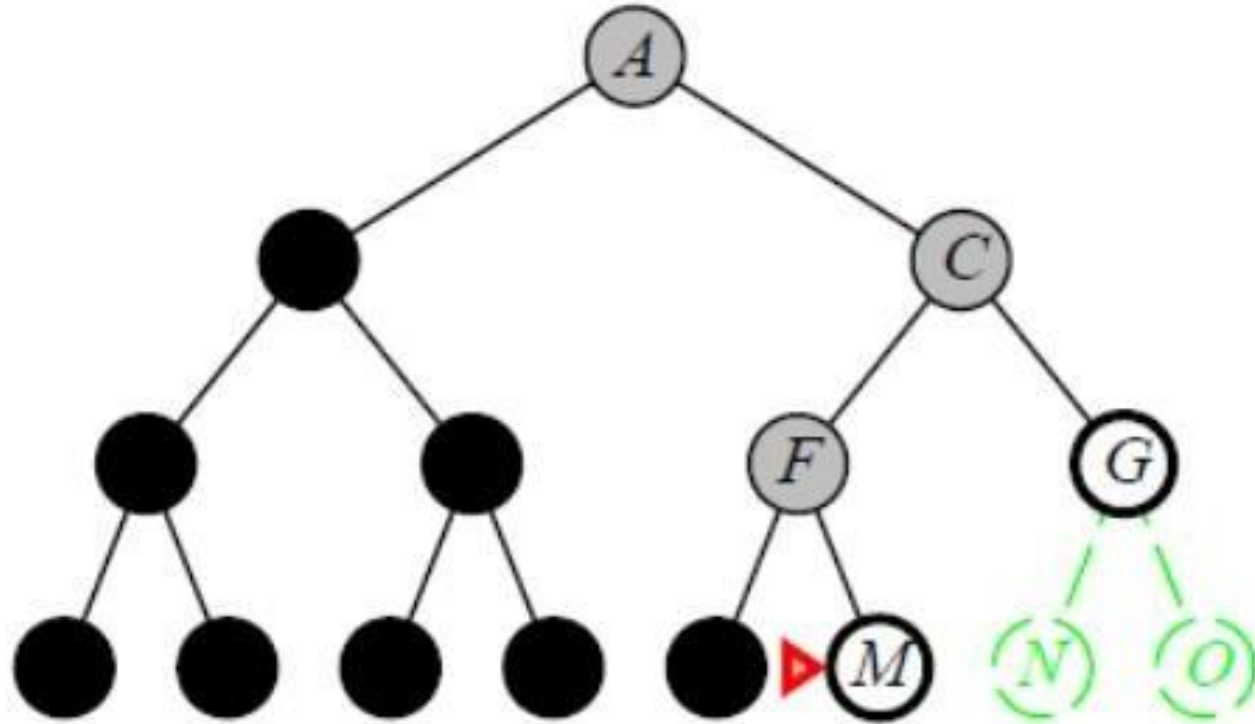
Depth-First Search



Depth-First Search



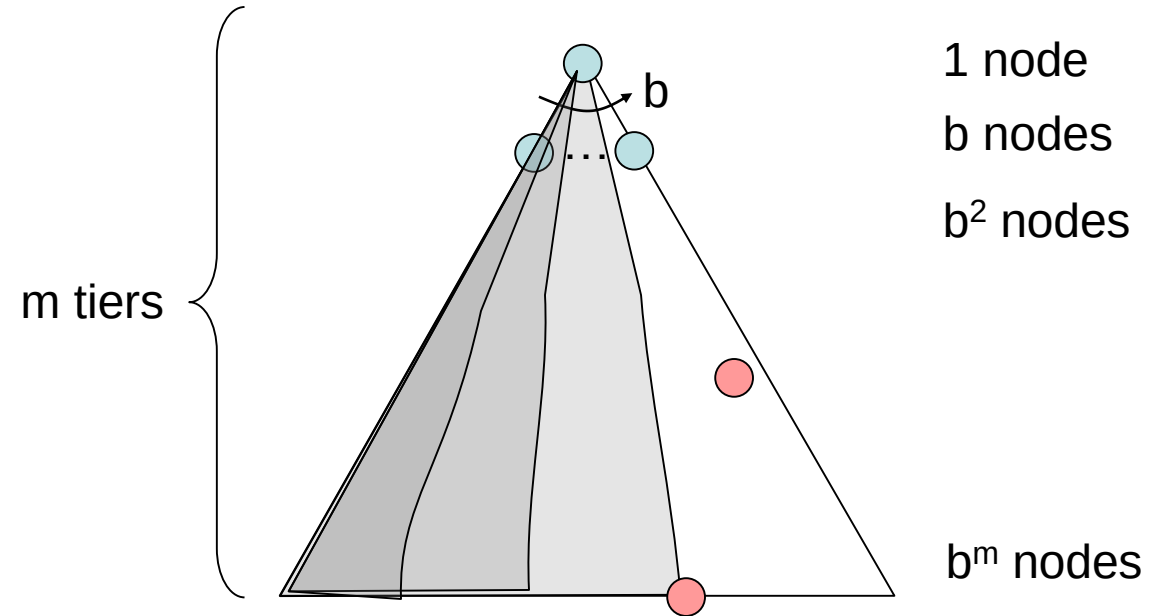
Depth-First Search



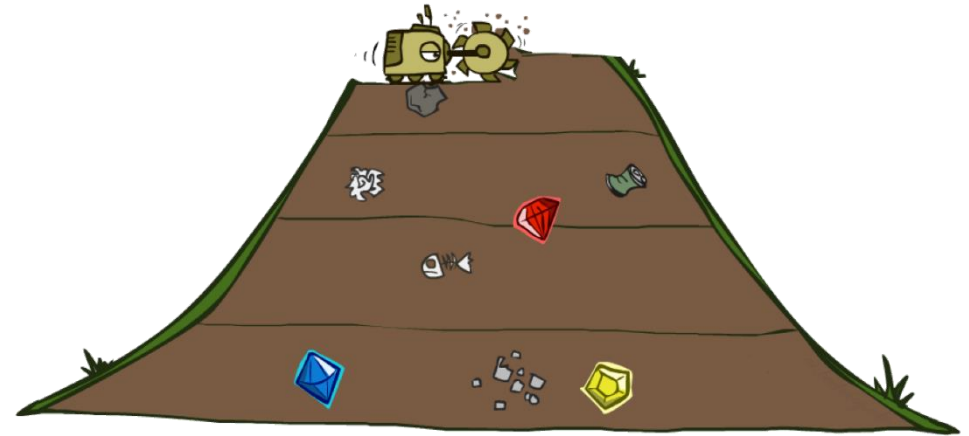
- Goal (M) is found

Depth-First Search (DFS) Properties

- What nodes DFS expand?
 - Some left prefix of the tree.
 - Could process the whole tree!
 - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
 - Only has siblings on path to root, so $O(bm)$
- Is it complete?
 - m could be infinite, so only if we prevent that
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost



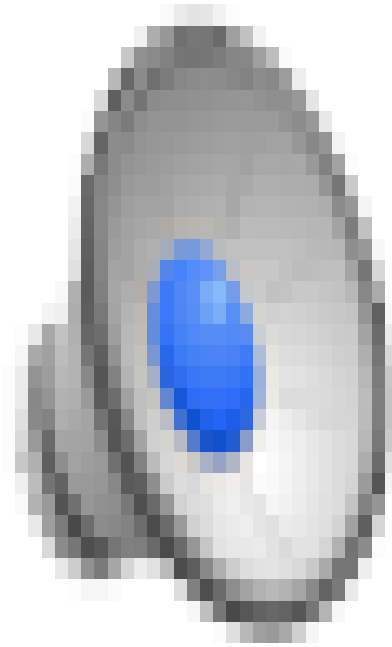
Quiz: DFS vs BFS



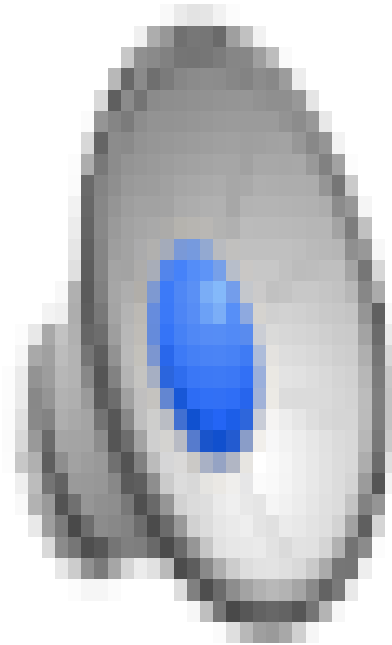
Quiz: DFS vs BFS

- When will BFS outperform DFS?
- When will DFS outperform BFS?

Video of Demo Maze Water DFS/BFS (part 1)



Video of Demo Maze Water DFS/BFS (part 2)



Depth-Limited Search

- The failure of depth-first search in infinite state spaces can be alleviated by supplying depth-first search with a predetermined depth limit ℓ .
- Nodes at depth ℓ are treated as if they have no successors.
- This approach is called **depth-limited search**.
- The depth limit solves the infinite-path problem.
- Unfortunately, it also introduces an additional source of incompleteness if we choose $\ell < d$, that is, the shallowest goal is beyond the depth limit.
- Depth-limited search will also be non-optimal if we choose $\ell > d$.
- Its time complexity is $O(b^\ell)$ and its space complexity is $O(b\ell)$.
- Depth-first search can be viewed as a special case of depth-limited search with $\ell = \infty$.

Depth-Limited Search

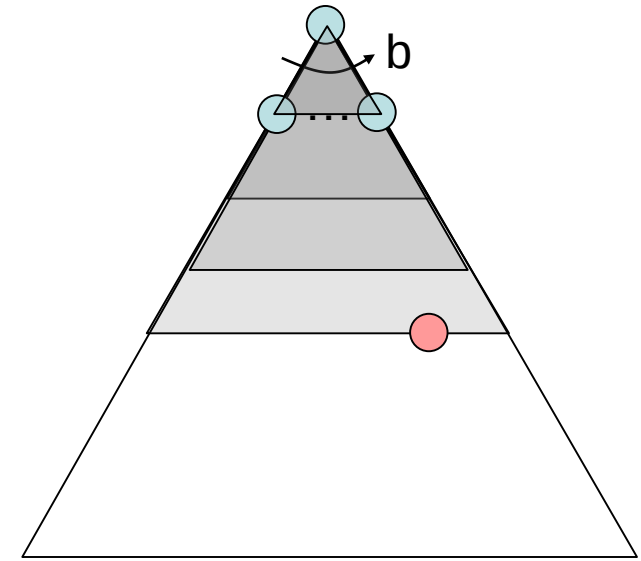
Recursive implementation

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff  
  RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)
```

```
function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff  
  cutoff-occurred?  $\leftarrow$  false  
  if GOAL-TEST(problem, STATE[node]) then return node  
  else if DEPTH[node] = limit then return cutoff  
  else for each successor in EXPAND(node, problem) do  
    result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)  
    if result = cutoff then cutoff-occurred?  $\leftarrow$  true  
    else if result  $\neq$  failure then return result  
  if cutoff-occurred? then return cutoff else return failure
```

Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally most work happens in the lowest level searched, so not so bad!
 - Branching factor 10, solution 5 deep:
 - BFS: $10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$
 - IDS: $50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$



Iterative Deepening Depth-First Search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution
  inputs: problem, a problem
  for depth  $\leftarrow$  0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
  end
```

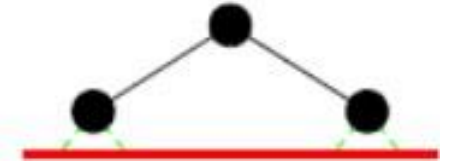
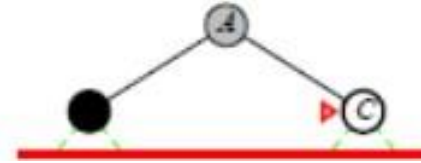
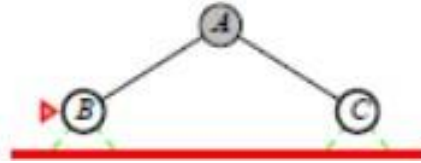
Iterative deepening search $l = 0$

Limit = 0



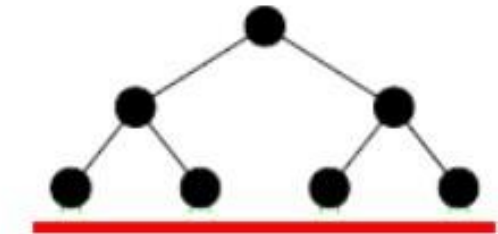
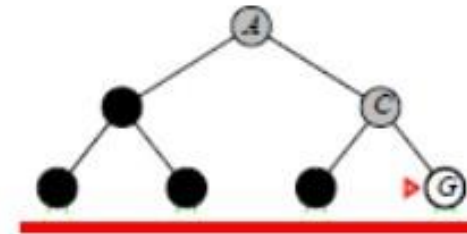
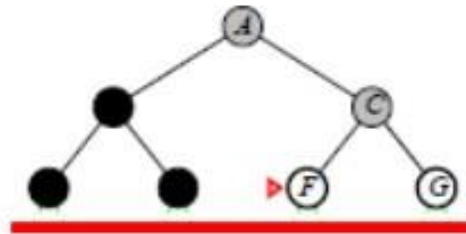
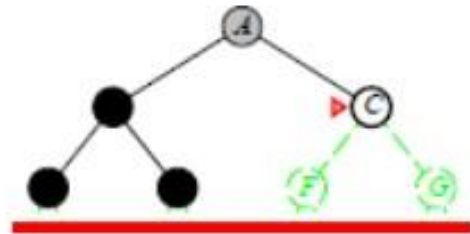
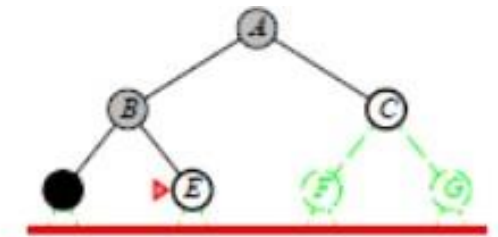
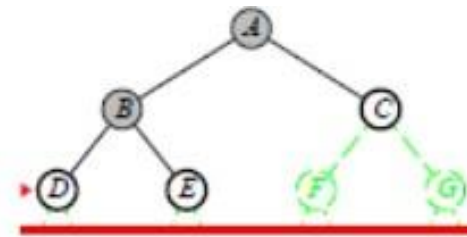
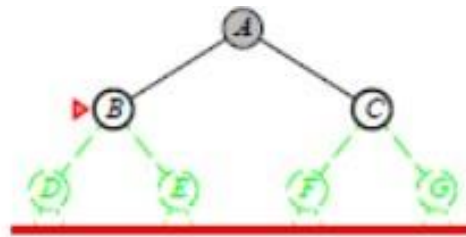
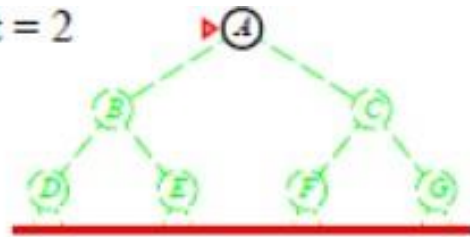
Iterative deepening search $l = 1$

Limit = 1



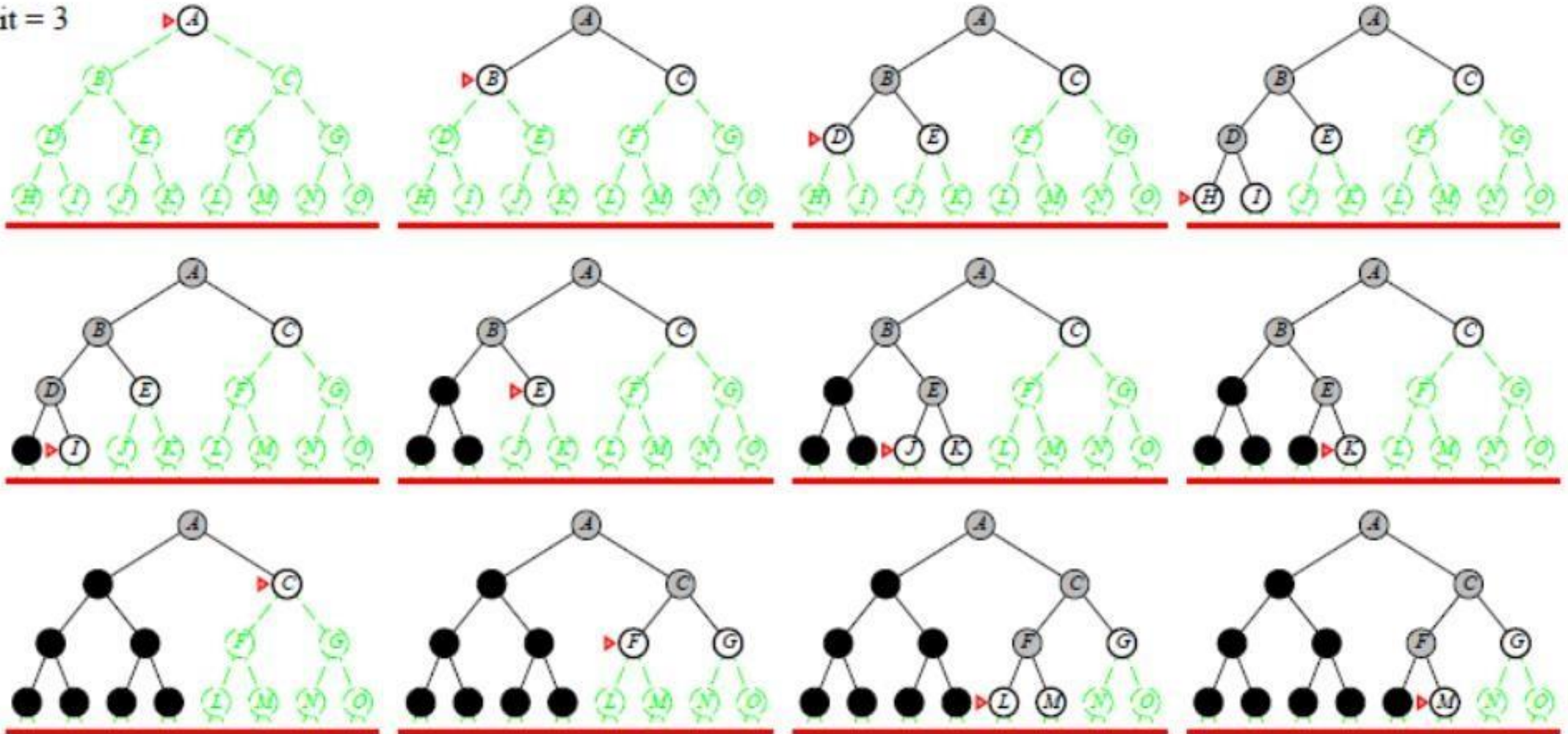
Iterative deepening search $l = 2$

Limit = 2



Iterative deepening search $l = 3$

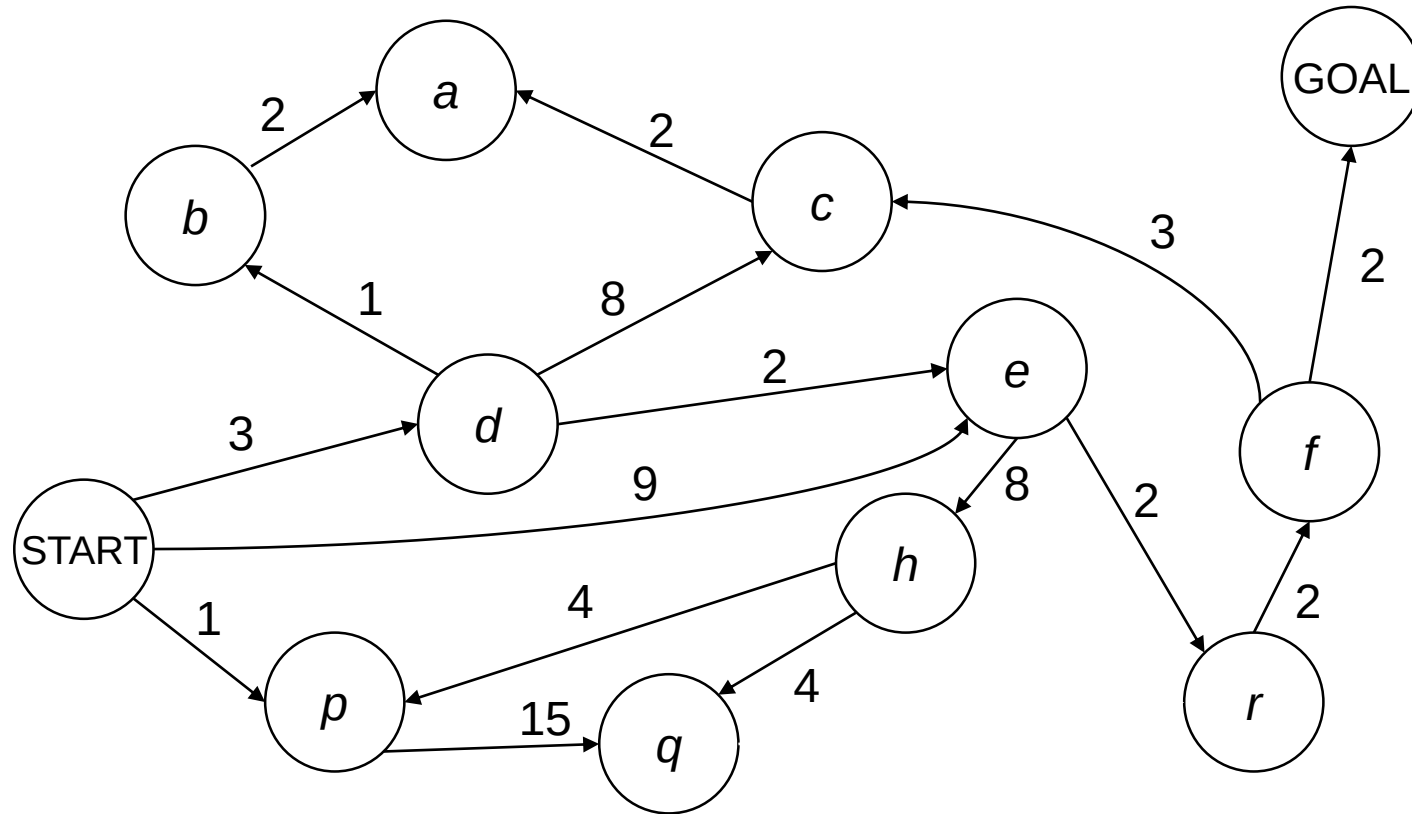
Limit = 3



Properties of iterative deepening search

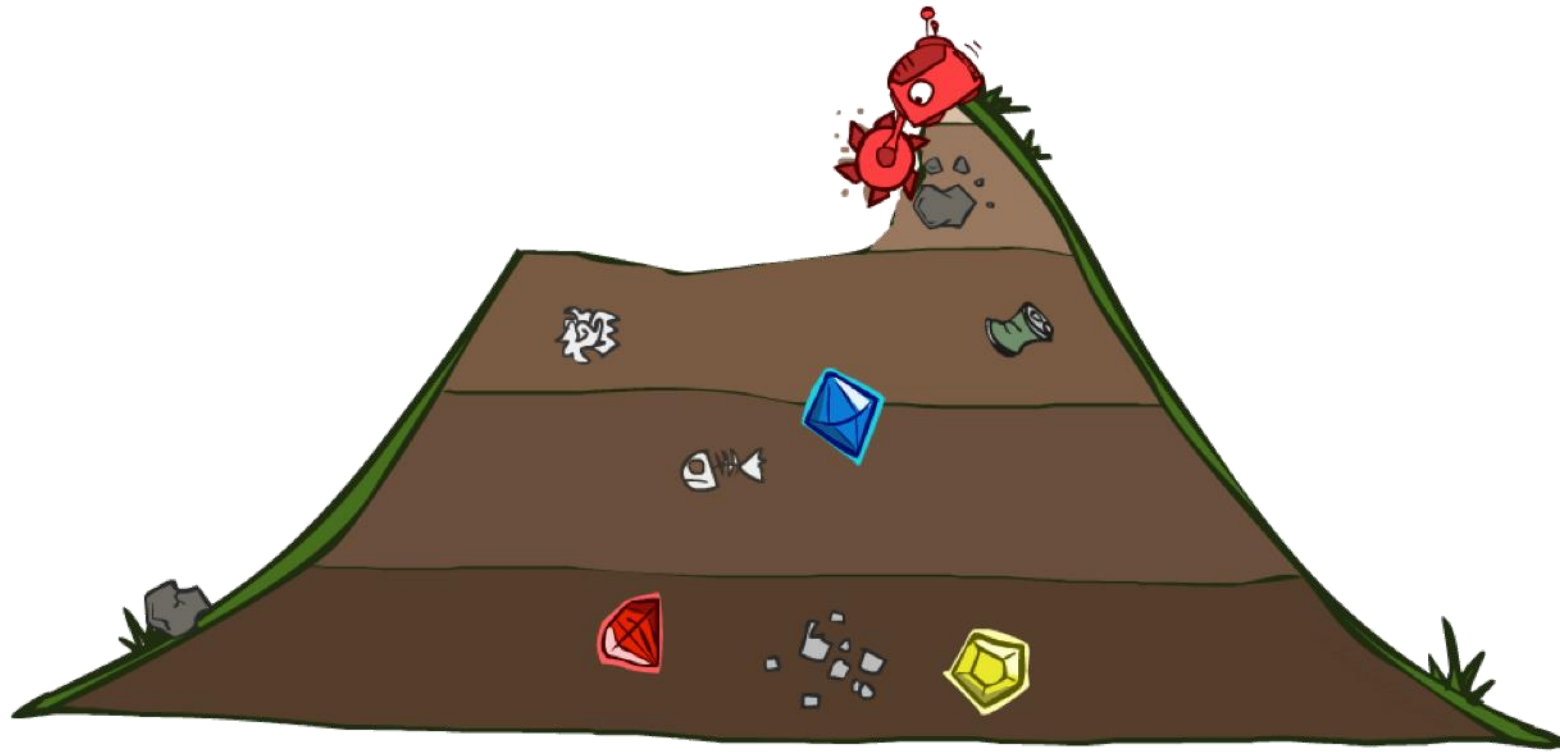
Complete?	Yes	
Time?	$O(b^d)$	
Space?	$O(bd)$	linear space
Optimal?	Yes	if step cost = 1

Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.

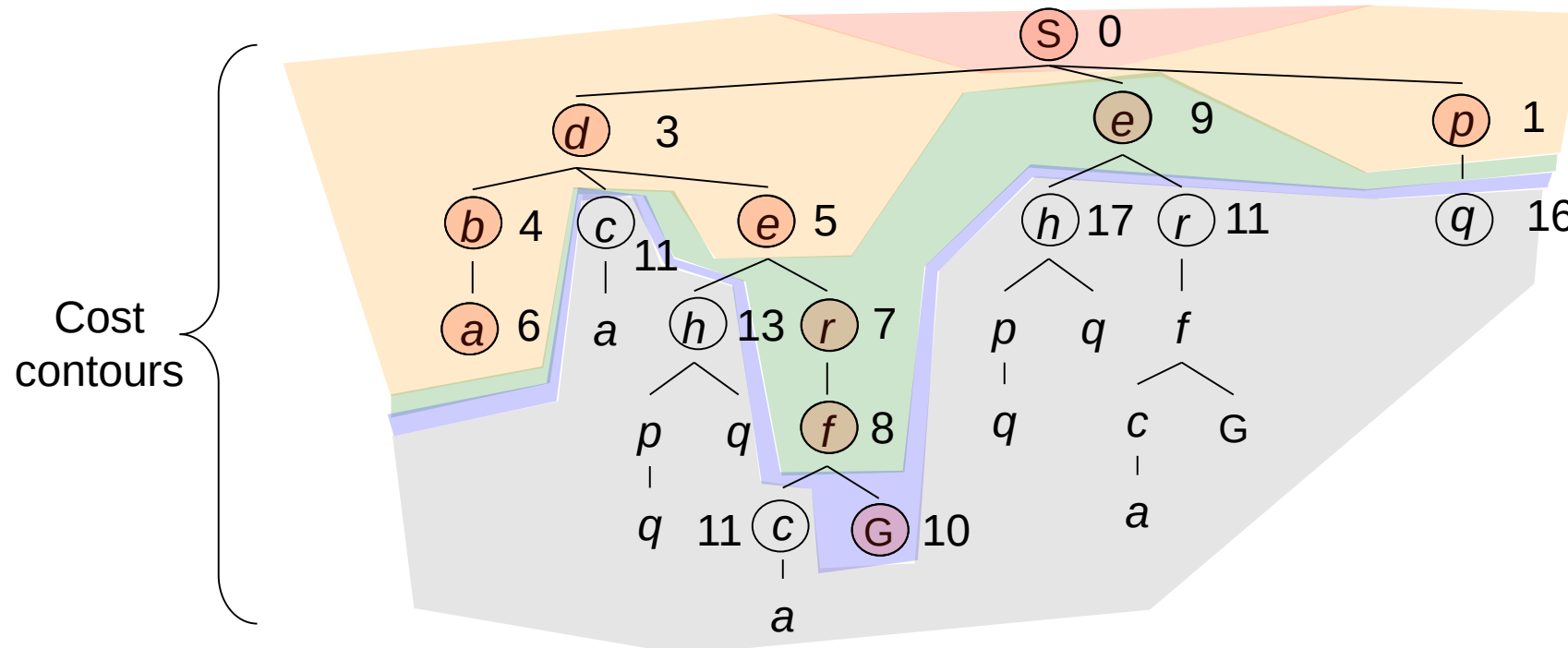
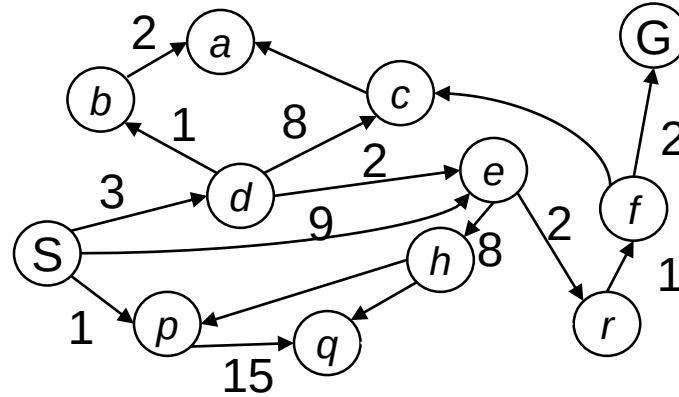
Uniform Cost Search



Uniform Cost Search

*Strategy: expand a
cheapest node first:*

*Fringe is a priority queue
(priority: cumulative cost)*



Uniform-Cost Search

- When all ***step costs*** are equal, **breadth-first search** is **optimal** because it always expands the shallowest unexpanded node.
 - In general, it is not optimal when step costs are different.
 - By a simple extension, we can find an algorithm that is **optimal** with ***any step-cost function***.
- **Uniform-cost search** expands node n with the ***lowest path cost $g(n)$*** .
- This is done by storing the ***frontier*** as a ***priority queue*** ordered by g .
- **Uniform-cost search** is equivalent to breadth-first if step costs are all equal

Uniform-Cost Search

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

Uniform Cost Search (UCS) Properties

■ What nodes does UCS expand?

- Processes all nodes with cost less than cheapest solution!
- If that solution costs C^* and arcs cost at least ε , then the “effective depth” is roughly C^*/ε
- Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)

■ How much space does the fringe take?

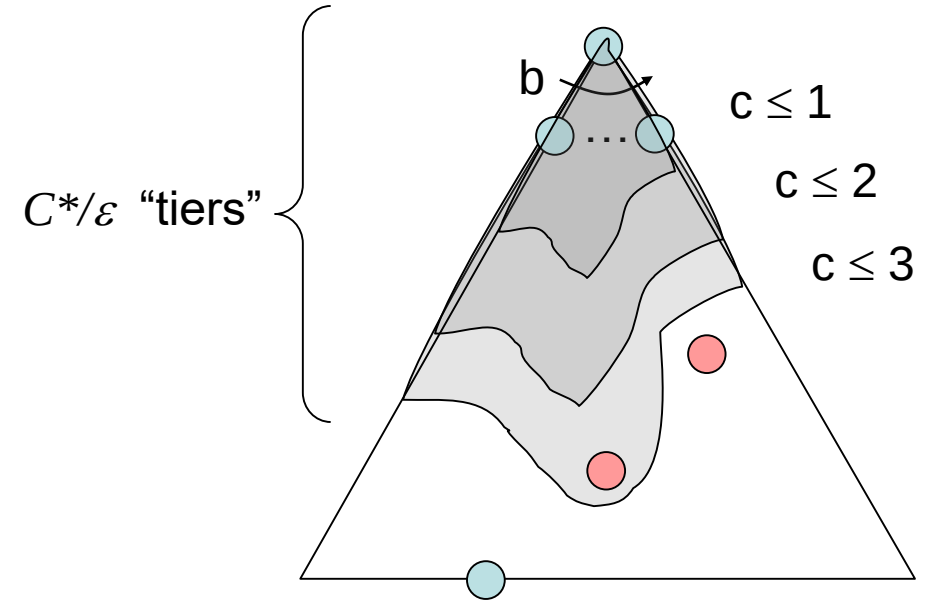
- Has roughly the last tier, so $O(b^{C^*/\varepsilon})$

■ Is it complete?

- Assuming best solution has a finite cost and minimum arc cost is positive, yes!

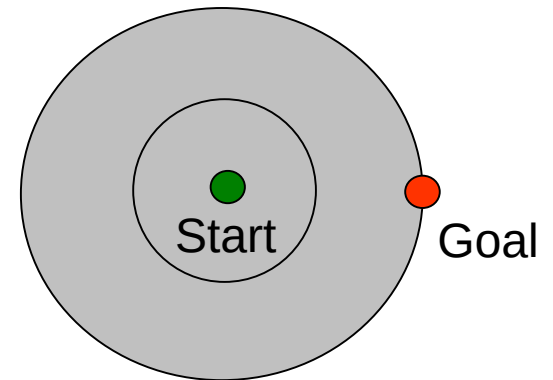
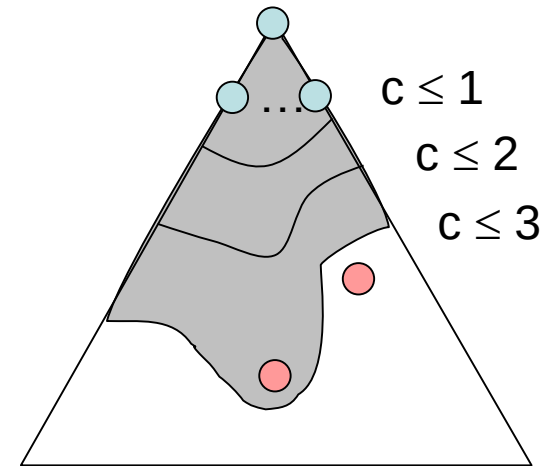
■ Is it optimal?

- Yes! (Proof next lecture via A^*)



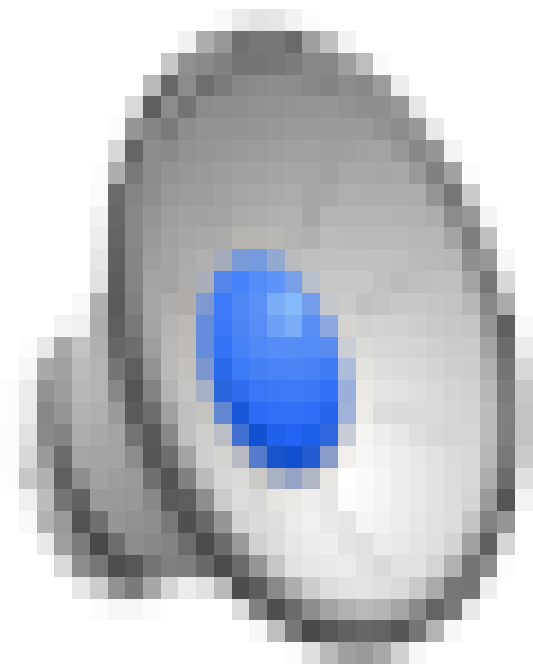
Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
- We'll fix that soon!

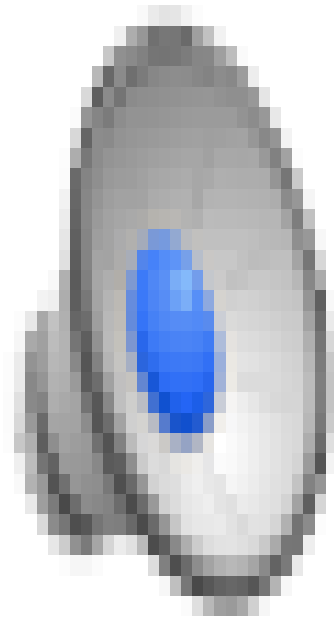


[Demo: empty grid UCS (L2D5)]
[Demo: maze with deep/shallow water DFS/BFS/UCS (L2D7)]

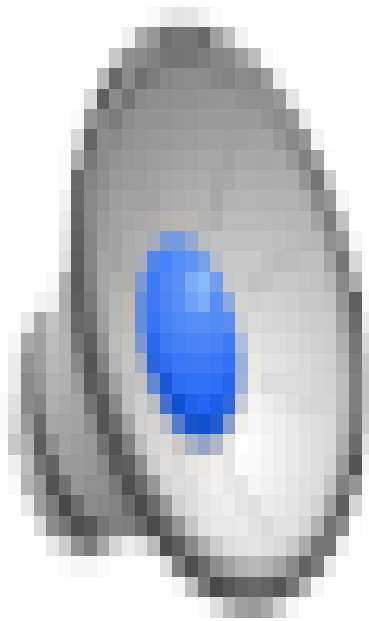
Video of Demo Contours UCS Pacman Small Maze



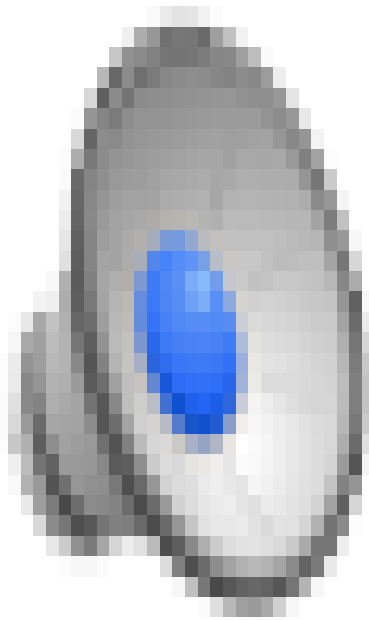
Video of Demo Empty UCS



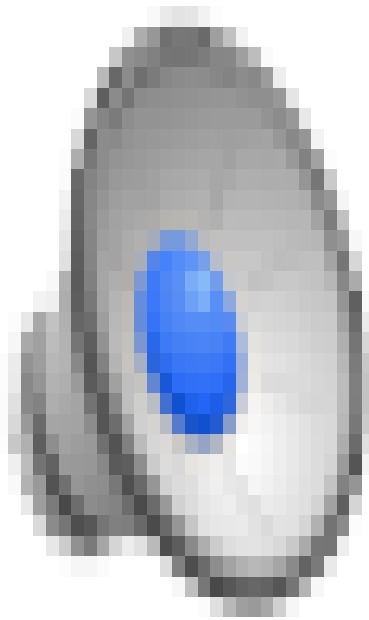
Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 1)



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 2)



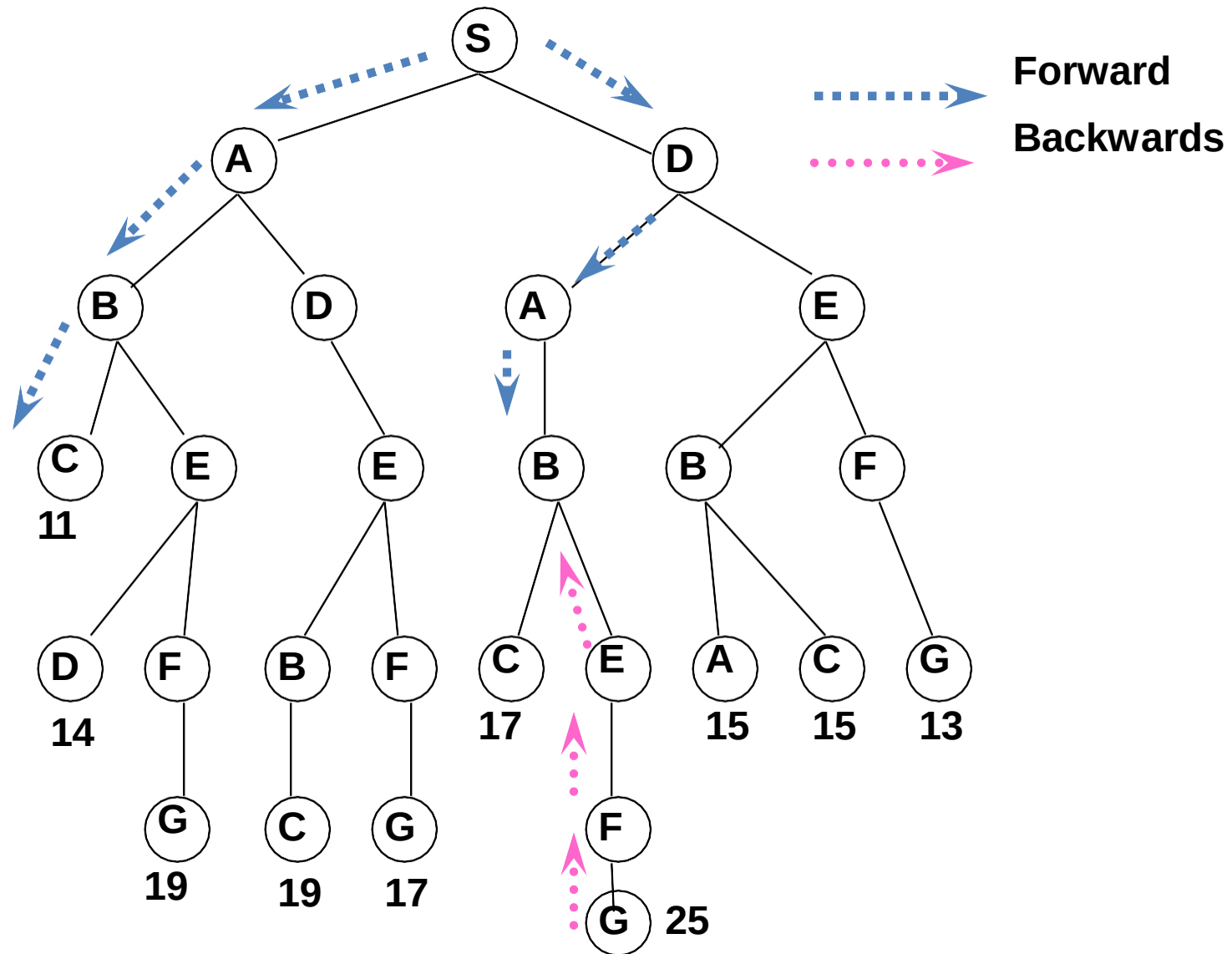
Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 3)



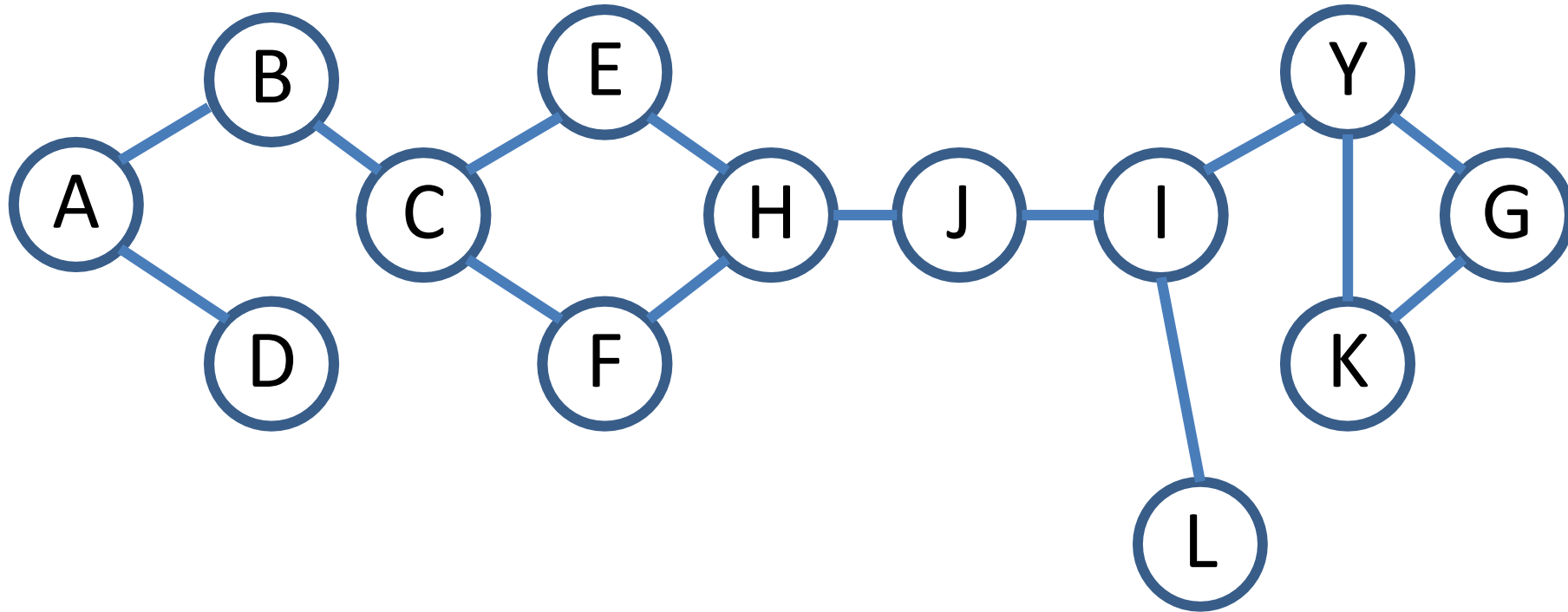
6- Bi-directional Search

- Both search **forward** from **initial state**, and **backwards** from **goal**.
- Stop when the two searches meet in the middle.
- Motivation: $b^{d/2} + b^{d/2}$ is much less than b^d
- Implementation
 - Replace the goal test with a check to see whether the frontiers of the two searches intersect, if yes $\rightarrow$ solution is found

6- Bi-directional Search



6- Bi-directional Search Example Using BFS



6- Bi-directional Search

Search simultaneously from two directions

- Forward from the initial and backward from the goal state, until they meet in the middle (i.e., if a node exists in the fringe of the other).
- The idea is to have $(b^{d/2} + b^{d/2})$ instead of b^d , which is much less

May lead to substantial savings (if it is applicable), but it has several limitations

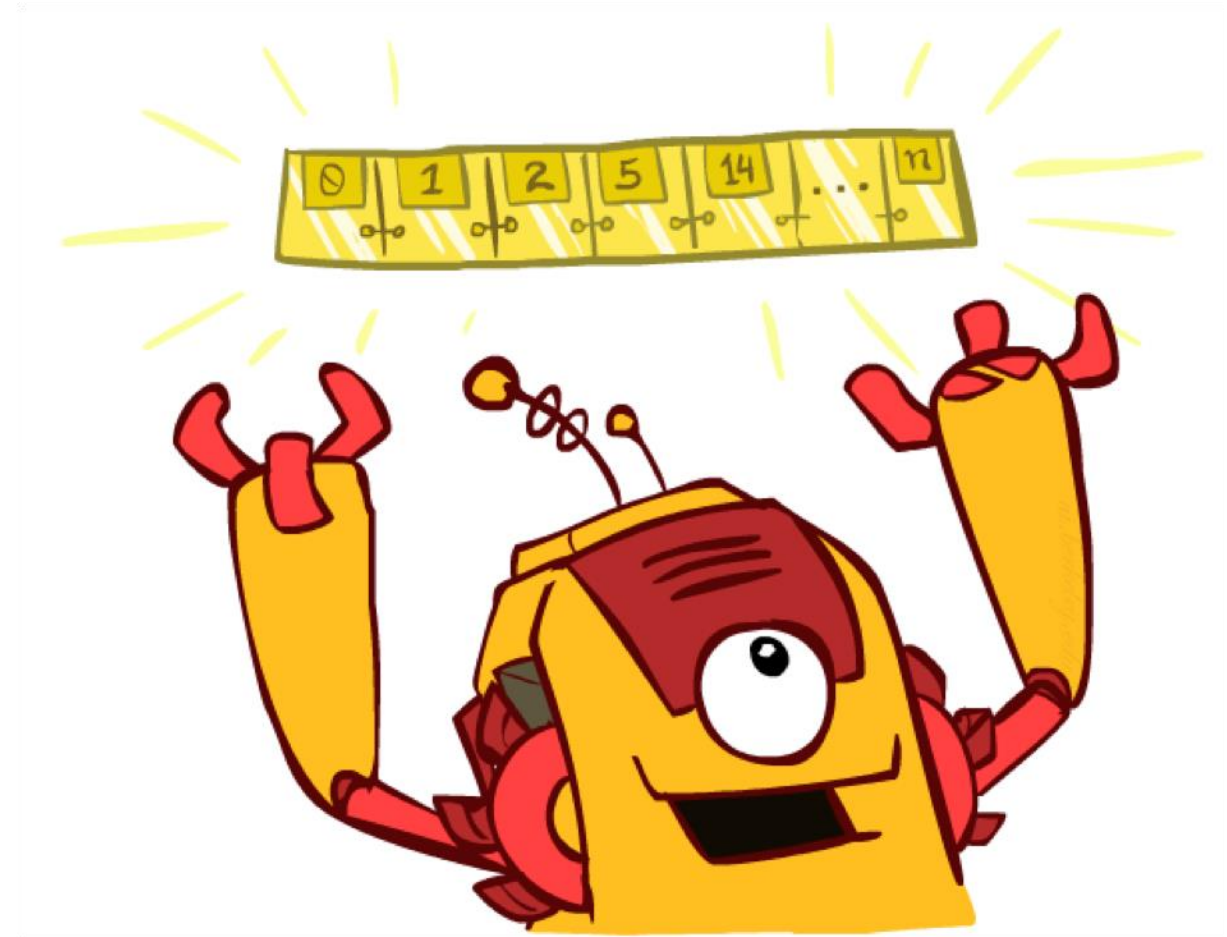
- Predecessors must be generated, which is not always possible
- Search must be coordinated between the two searches
- One search must keep all nodes in memory

Time Complexity	$b^{d/2}$
Space Complexity	$b^{d/2}$
Completeness	yes (b finite, breadth-first for both directions)
Optimality	yes (all step costs identical, breadth-first for both directions)

b	branching factor
d	tree depth

The One Queue

- All these search algorithms are the same except for fringe strategies
 - Conceptually, all fringes are priority queues (i.e. collections of nodes with attached priorities)
 - Practically, for DFS and BFS, you can avoid the $\log(n)$ overhead from an actual priority queue, by using stacks and queues
 - Can even code one implementation that takes a variable queuing object



Comparing uninformed search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*

- b is the branching factor; d is the depth of the shallowest solution;
- m is the maximum depth of the search tree; l is the depth limit.
- Superscripts:
 - a complete if b is finite;
 - b complete if step costs $\geq \epsilon$ for positive ϵ ;
 - c optimal if step costs are all identical;